CSE 6230:
HPC Tools and Applications

Lecture 13: Dense Matrix Algorithms

Helen Xu
hxu615@gatech.edu

Georgia Tech College of Computing
School of Computational Science and Engineering

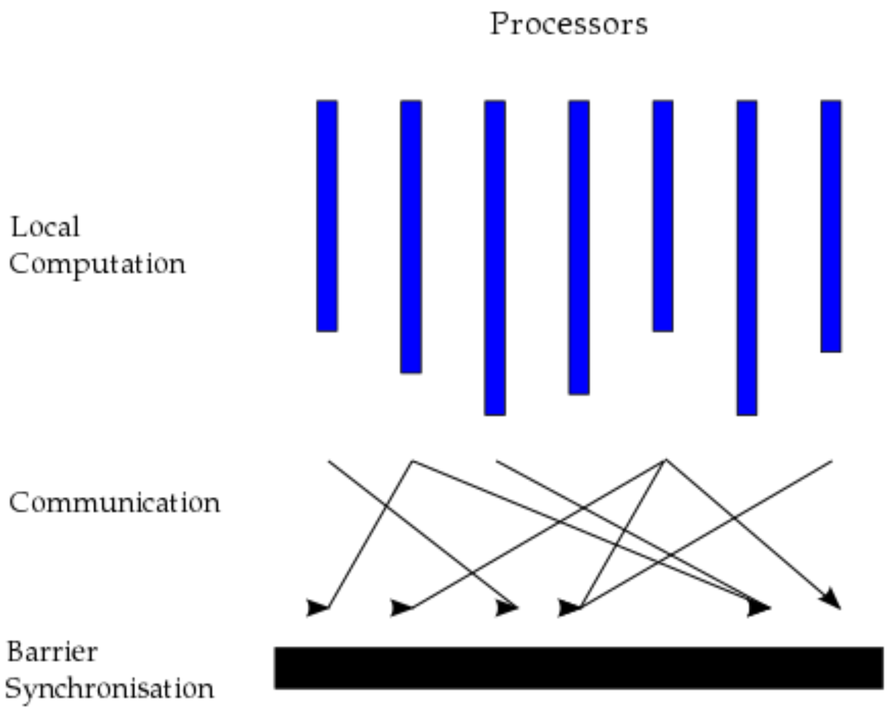
Slide credits to Prof. Srinivas Aluru

Bulk-Synchronous Parallel Model

Bulk-Synchronous Parallel (BSP) Computer

A BSP computer consists of the following:

- Components capable of **processing and/or local memory transactions** (i.e., processors),
- A **network** that routes messages between pairs of such components, and
- A hardware facility that allows for the **synchronization** of all or a subset of components.



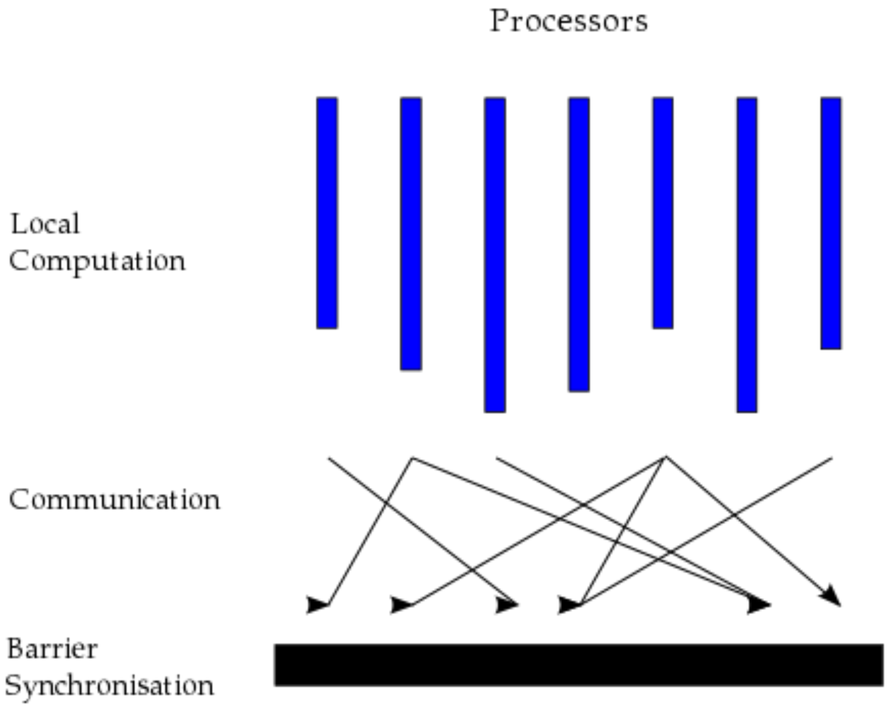
https://en.wikipedia.org/wiki/Bulk_synchronous_parallel

Leslie G. Valiant, “A bridging model for parallel computation,” Communications of the ACM, Volume 33 Issue 8, Aug. 1990

Bulk-Synchronous Parallel (BSP) Algorithms

A BSP computation proceeds in a series of global ***supersteps***, which consists of three components:

- **Concurrent computation**: every participating processor may perform local computations which may **overlap with communication.**
- **Communication**: processes exchange data.
- **Barrier synchronization**: When a process reaches this point (the *barrier*), it waits until all other processes have reached the same barrier.

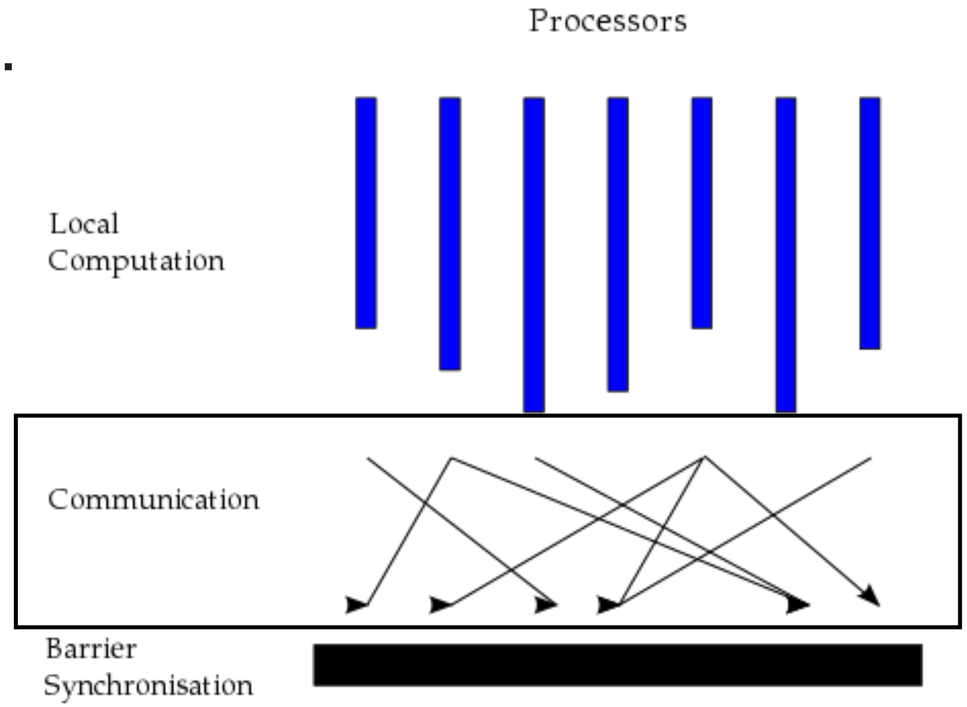


https://en.wikipedia.org/wiki/Bulk_synchronous_parallel

Leslie G. Valiant, “A bridging model for parallel computation,” Communications of the ACM, Volume 33 Issue 8, Aug. 1990

Communication in BSP

- It is difficult to determine the time any single communication action will complete since there are may be many simultaneous communications.
- The BSP model provides an **upper bound** on the time to communicate a set of data:
  - Let h be the number of incoming or outgoing messages in a superstep.
  - The ability of a communication network to deliver data is captured by some parameter g, such that it takes **time hg for a process to deliver h messages of size 1**.



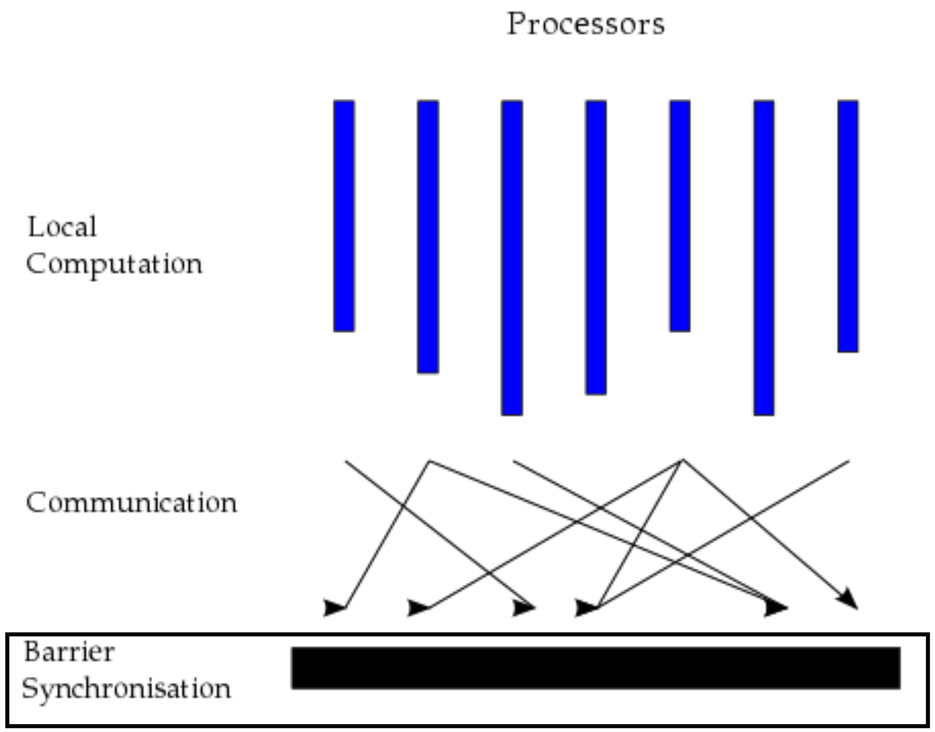
Ignores latency

https://en.wikipedia.org/wiki/Bulk_synchronous_parallel

Leslie G. Valiant, “A bridging model for parallel computation,” Communications of the ACM, Volume 33 Issue 8, Aug. 1990

Barriers in BSP

- The cost of **barrier synchronization** is denoted by ℓ.
- The cost of barriers is influenced by a couple of issues:
  - The cost imposed by the **variation in the completion time** of the different processes.
  - The **cost of reaching a globally consistent state** in all of the processors.



https://en.wikipedia.org/wiki/Bulk_synchronous_parallel

Leslie G. Valiant, “A bridging model for parallel computation,” Communications of the ACM, Volume 33 Issue 8, Aug. 1990

Analyzing Algorithms in the BSP Model

The cost of a superstep is the sum of three terms:

- The cost of the longest-running **computation**,
- The cost of global **communication** between the processors, and
- The cost of the barrier **synchronization** at the end of the superstep.

Cost for local computation in process i

Messages sent or received by process i

$$\max_{i=1}^{p}(w_i) + \max_{i=1}^{p}(h_i g) + \ell$$

Synchronization cost

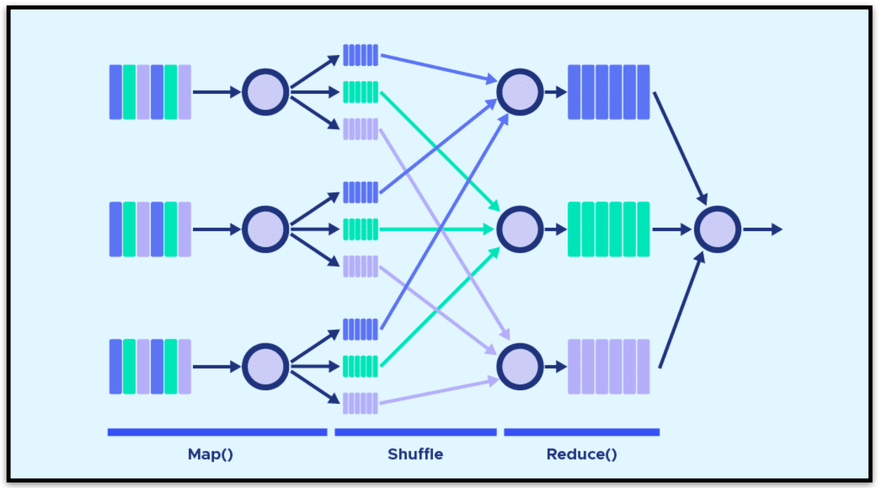
The cost of an entire BSP algorithm is the sum of the cost of **all the supersteps**.

https://en.wikipedia.org/wiki/Bulk_synchronous_parallel

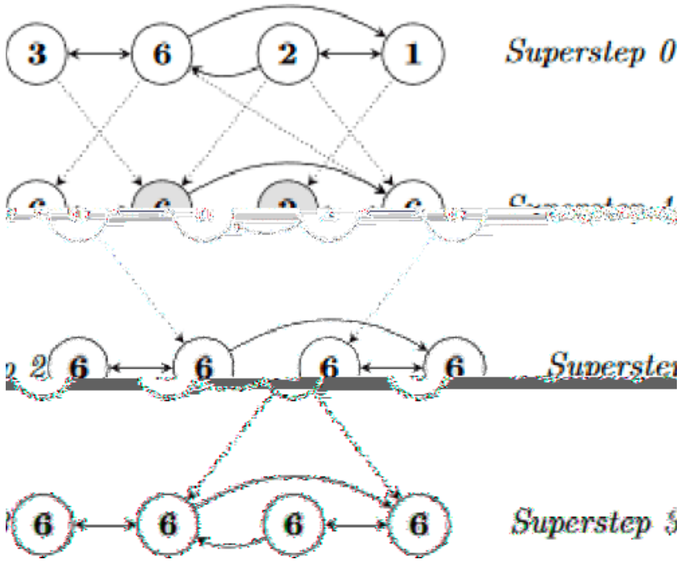
Leslie G. Valiant, “A bridging model for parallel computation,” Communications of the ACM, Volume 33 Issue 8, Aug. 1990

Applications of BSP

One of the main applications for BSP is Google's graph analysis platforms Pregel and MapReduce.



https://datascientest.com/en/mapreduce-how-to-use-it-for-big-data



https://www.semanticscholar.org/paper/Pregel%3A-a-system-for-large-scale-graph-processing-Malewicz-Austern/2d867297dfe0d3ce2ed5b1d0f2dff88cac46ee94

https://en.wikipedia.org/wiki/Bulk_synchronous_parallel

Recall: Latency and Bandwidth

Ignores topology

Similarly, we will analyze algorithms in rounds/supersteps according to the latency-bandwidth model from last class:

Communication cost of one round = latency + n * cost_per_word
= latency + n/bandwidth
= $\tau + n\mu$

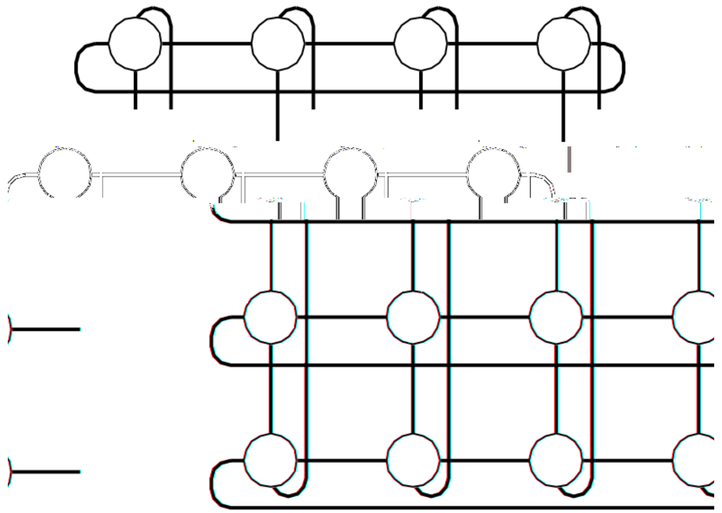
Total cost of one round = computation cost + communication cost

Total cost algorithm = Sum of costs over rounds

Matrix Algorithms

Matrix Algorithms

- Regular structure
- Well-suited to **static partitioning** of computation
- Computation typically partitioned based on the data
  - input, output, intermediate
- Typical data partitioning strategies for matrix algorithms
  - 1D and 2D block, cyclic, block-cyclic
- Easier to think logically about the algorithms on a 2D mesh of processors:



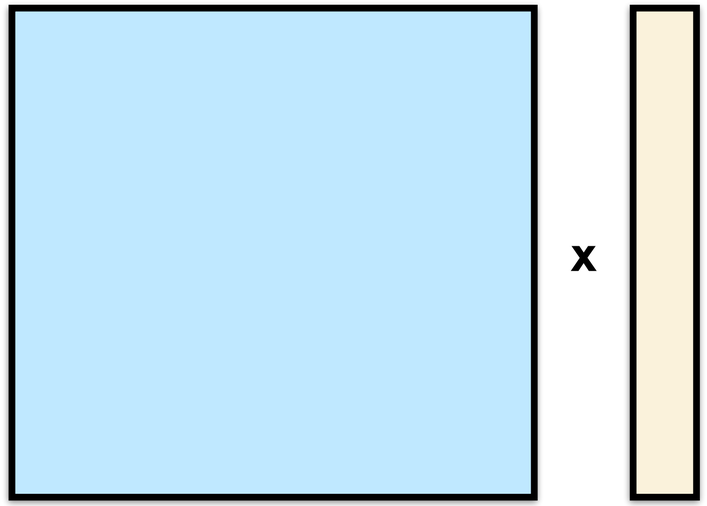
Matrix-Vector Multiplication

Problem:

- multiply a dense $n \times n$ matrix A with an $n \times 1$ vector x
- yield the $n \times 1$ result vector y

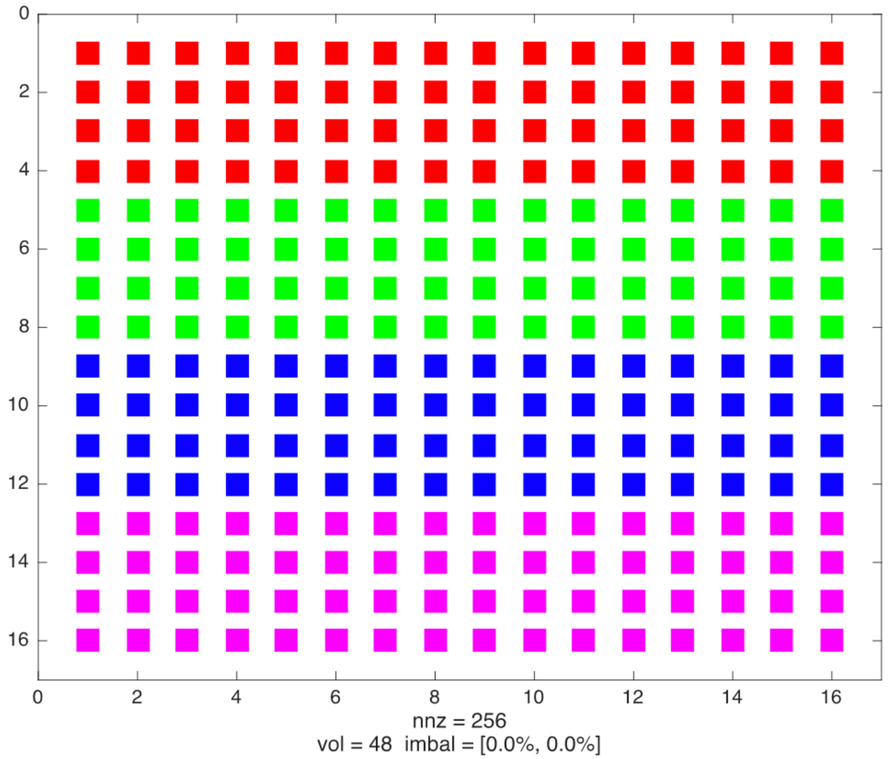
What is the serial time complexity?

- Serial algorithm requires n^2 multiplications and additions



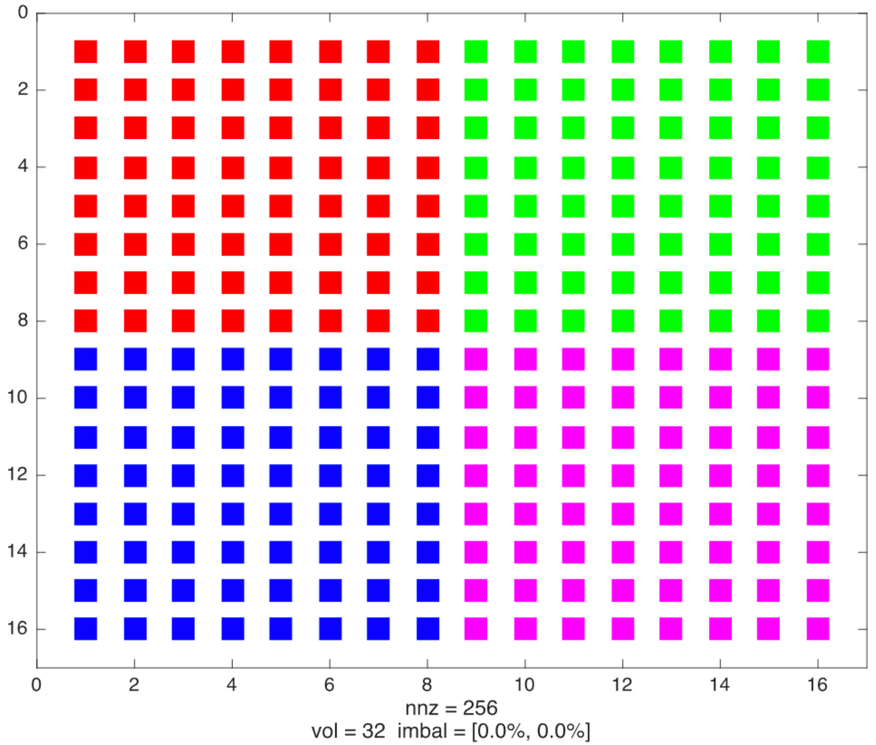
1D Partitioning for Matrix-Vector Multiply

- 1D partitioning splits up the matrix into **block rows** with at least one row on each processor.
- Given n rows, 1D partitioning can use at most n processors.



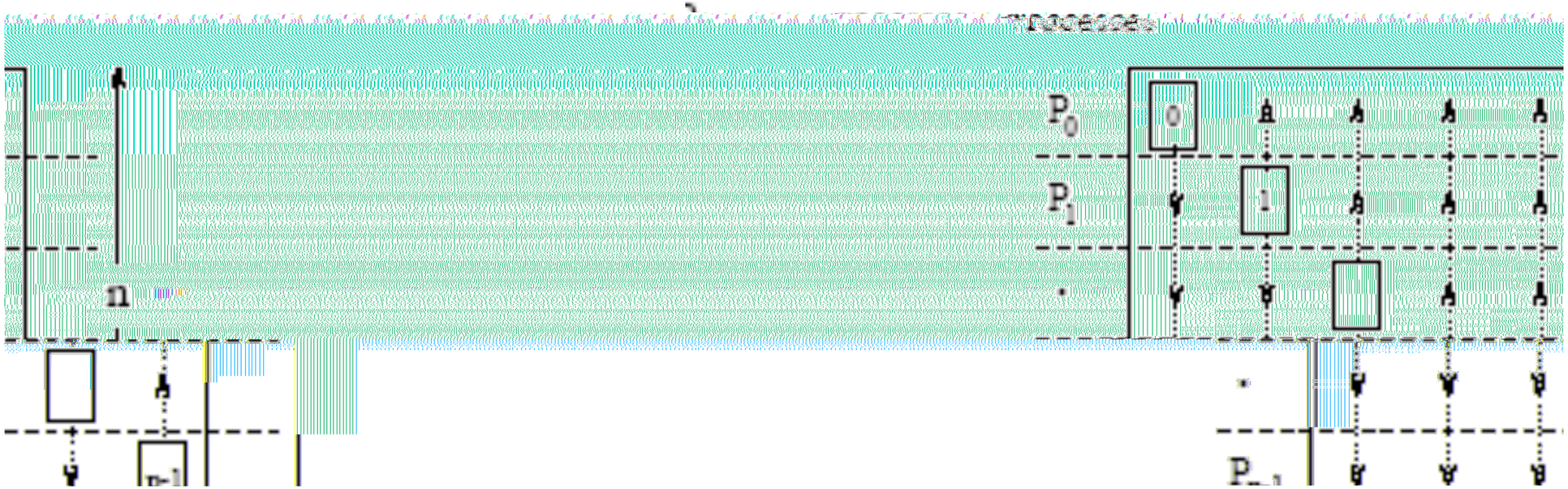
2D Partitioning for Matrix-Vector Multiply

- 2D partitioning splits up the matrix into **square sub-matrices** - at least 1 element per processor
- Theoretically, 2D partitioning can use up to n^2 processors (equal to number of elements)



Analysis of 1D Partitioning

- First, consider simple case: $p = n$
- n x n matrix is partitioned among p processors
  - *Each processor stores a complete row of the matrix*
- n x 1 vector x is also partitioned in a conformal manner
  - Each process owns one element of x



Analysis of 1D Partitioning

: Use AllGather to distribute all of x to each
ssor

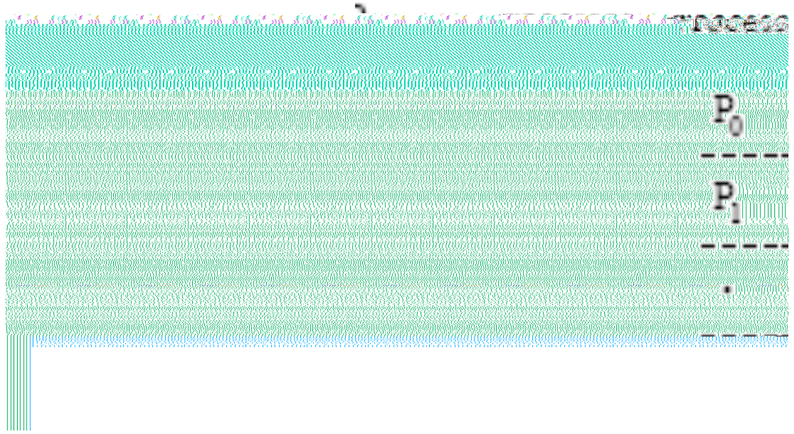
$\log n + \mu \cdot n)$

$\mu n > n$, so time is basically the cost of allgather

: Processor P_i computes $y[i] = \sum_{j=0}^{n-1} A[i,j] \cdot x[j]$

(n)

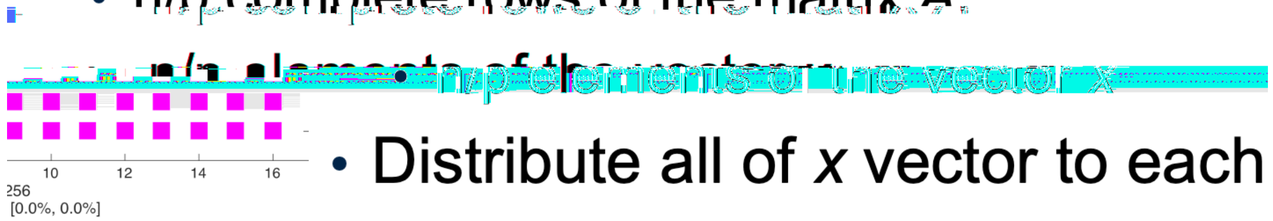
- Step 1
proces
$O(\tau$
- Step 2
O

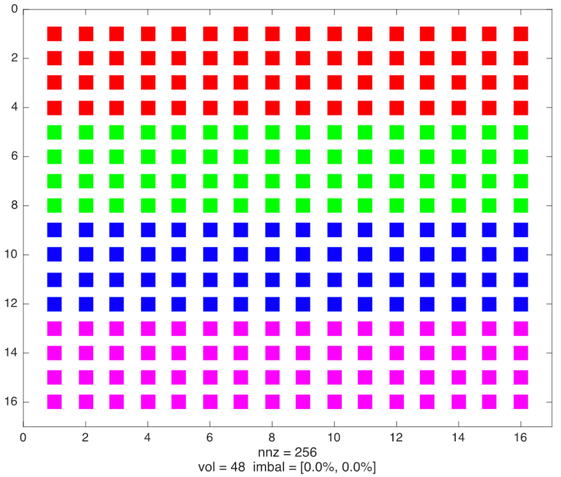


Analysis of 1D Partitioning

For $n>p$

- Initially, each processor stores
  - n/p complete rows of the matrix A
  - n/p elements of the vector x





- Distribute all of x vector to each
  - AllGather among p processors, messages o
- On each processor: n/p local dot products on vectors (l length n

rows) of

$p + u \cdot n)$

- Step 1: Com g n
- Ste

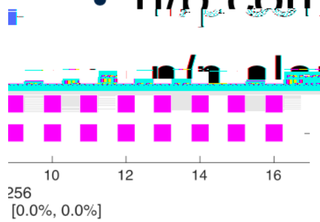
What is the communication time?

What is the computation time?

Analysis of 1D Partitioning

For $n>p$

- Initially, each processor stores
  - n/p complete rows of the matrix A
  - n/p elements of the vector x
- Distribute all of x vector to each
  - AllGather among p processors, messages
- On each processor: n/p local dot products on vectors (l length n



- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu\, m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu\, m\, p)$

rows) of

$\tau + \mu \cdot n)$

- Step 1: Com

$g\, n$

What is the communication time?

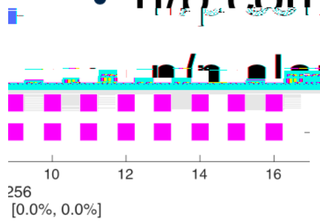
- Ste

What is the computation time?

Analysis of 1D Partitioning

For $n>p$

- Initially, each processor stores
  - n/p complete rows of the matrix A
  - n/p elements of the vector x
- Distribute all of x vector to each
  - AllGather among p processors, messages
- On each processor: n/p local dot products on vectors (l length n



- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu\, m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu\, m\, p)$

rows) of

$p + \mu \cdot n)$

- Step 1: Comm. Time = $O\left(\tau \log p + \mu \cdot \frac{n}{p} \cdot p\right) = O(\tau \log p$

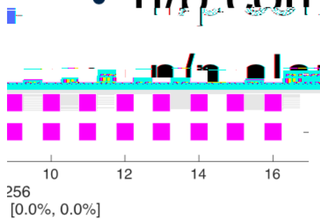
- Ste

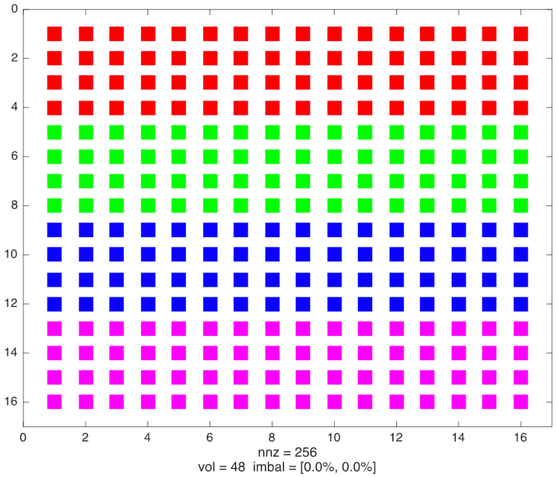
What is the computation time?

Analysis of 1D Partitioning

For $n>p$

- Initially, each processor stores
  - n/p complete rows of the matrix A
  - n/p elements of the vector x





- Distribute all of x vector to each
  - AllGather among p processors, messages o
- On each processor: n/p local dot products on vectors (l length n

rows) of

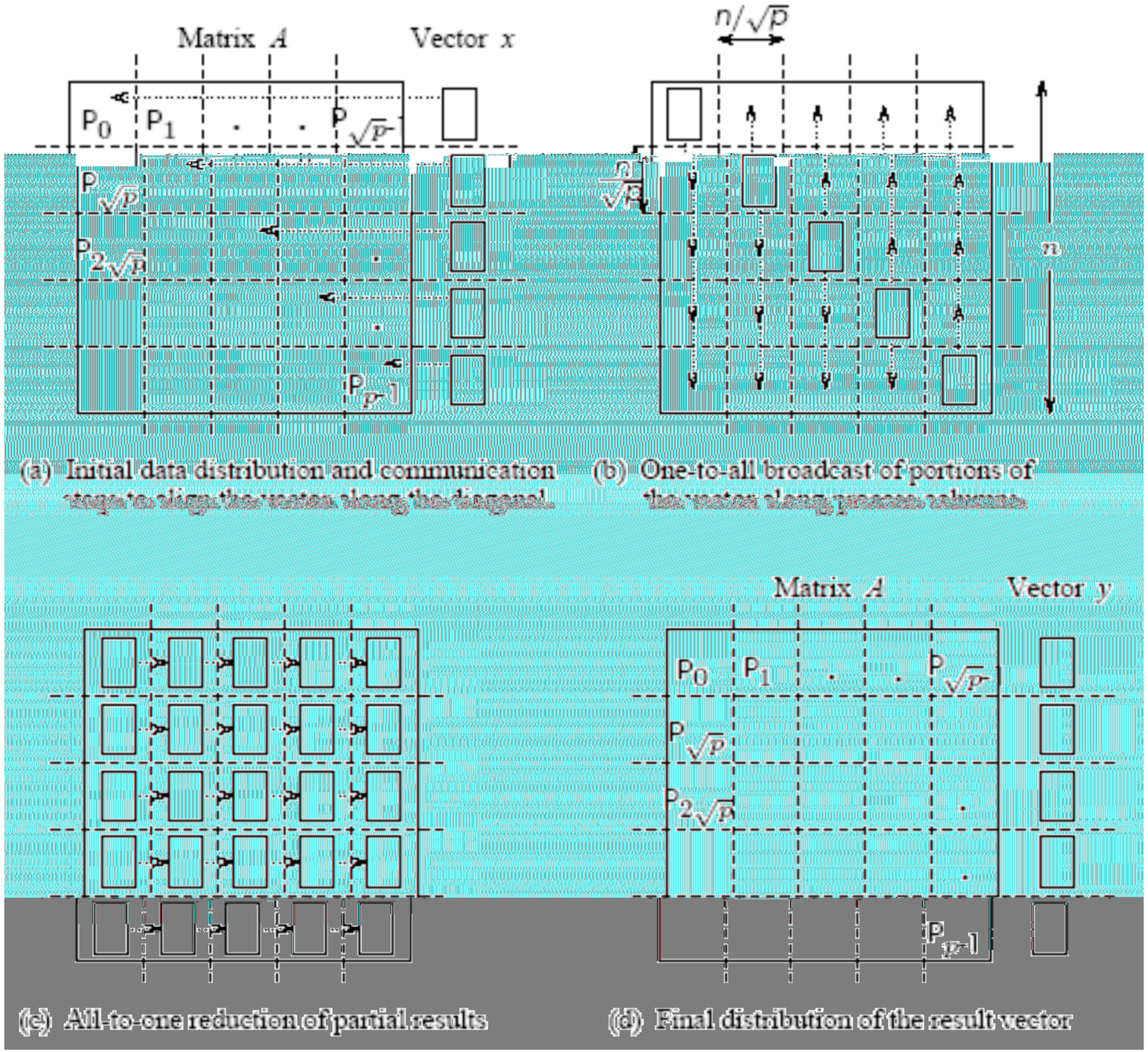
$\tau + \mu \cdot n)$

- Step 1: Comm. Time = $O\left(\tau \log p + \mu \cdot \frac{n}{p} \cdot p\right) = O(\tau \log p$
- Step 2: Computation time = $O\left(\frac{n^2}{p}\right)$

Analysis of 2D Partitioning

- Consider : $p = n^2$
- n x n matrix is partitioned among $p = n^2$ processors (virtual grid topology)
  - *Each processor owns a single element*
- n x 1 vector x is distributed in the last column of n processors

What is a good algorithm given this distribution?


Matrix A
Vector x
$n/\sqrt{p}$
P_0
P_1
$P_{\sqrt{p}-1}$
$P_{\sqrt{p}}$
$P_{2\sqrt{p}}$
P_{p-1}
n
(a) Initial data distribution and communication steps to align the vector along the diagonal
(b) One-to-all broadcast of portions of the vector along processor columns
Matrix A
Vector y
P_0
P_1
$P_{\sqrt{p}-1}$
$P_{\sqrt{p}}$
$P_{2\sqrt{p}}$
P_{p-1}
(c) All-to-one reduction of partial results
(d) Final distribution of the result vector

Analysis of 2D Partitioning

$p = n^2$

- Algorithm
  - align vector along the main diagonal: one-to-one communication
  - Broadcast vector element to n processors in column: one-to-all broadcast
  - Local multiplication
  - Sum partial y values in each row: all-to-one reduction

$\tau + \mu$

$(\tau + \mu)\lg n$

1

$(\tau + \mu)\lg n + \lg n$

- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu m p)$

Analysis of 2D Partitioning

$p = n^2$

- Algorithm
  - align vector along the main diagonal: one-to-one communication
  - Broadcast vector element to n processors in column: one-to-all broadcast
  - Local multiplication
  - Sum partial y values in each row: all-to-one reduction
- Communication time = $O(\tau \log n + \mu \cdot \log n)$
- Computation time = $O(\log n)$

$\tau + \mu$

$(\tau + \mu)\lg n$

1

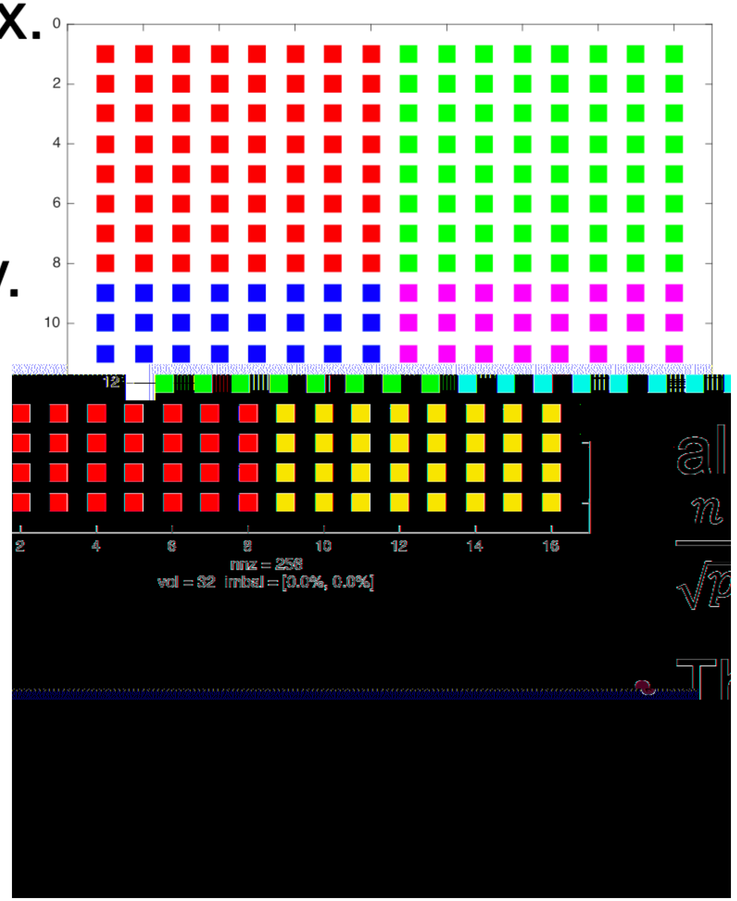
$(\tau + \mu)\lg n + \lg n$

From reduction

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

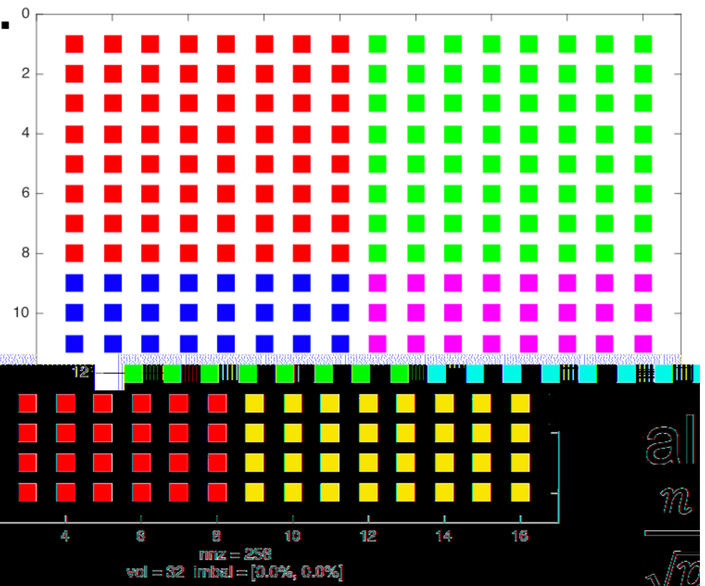
- When using fewer than n^2 processors, each process owns an $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$ block of the matrix.
- The vector is distributed in portions of $\frac{n}{\sqrt{p}}$ elements in the last processor-column only.



2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- When using fewer than n^2 processors, each process owns an $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$ block of the matrix.
- The vector is distributed in portions of $\frac{n}{\sqrt{p}}$ elements in the last processor-column only.
- In this case, the message sizes for the scatter, gather, and reduction are all $\frac{n}{\sqrt{p}}$

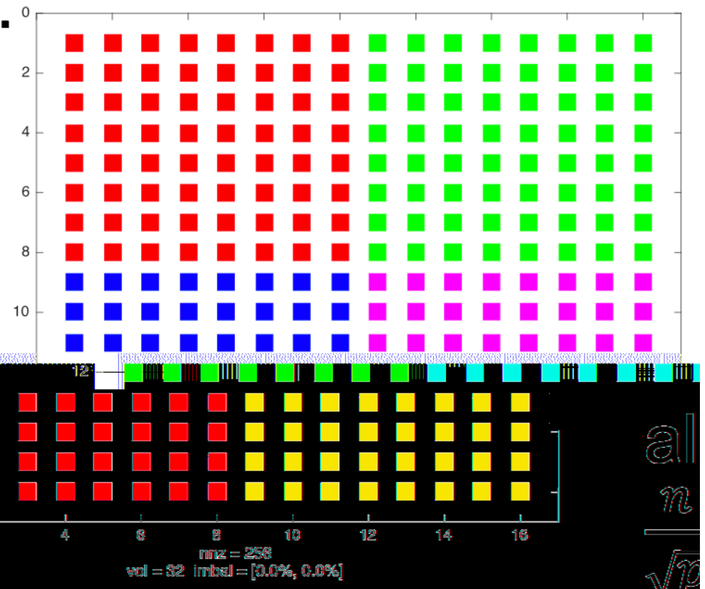


Part of the vector

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- When using fewer than n^2 processors, each process owns an $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$ block of the matrix.
- The vector is distributed in portions of $\frac{n}{\sqrt{p}}$ elements in the last processor-column only.
- In this case, the message sizes for the scatter, broadcast, and reduction are all $\frac{n}{\sqrt{p}}$.
- The computation is a product of an $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$ submatrix with a vector of length $\frac{n}{\sqrt{p}}$.



Run serial algorithm on the submatrix on each processor

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal:

What is the communication time?

How many vector elts?

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$

assuming direct connection

message size $O(n/\sqrt{p})$

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$
  - Broadcast vector elements to processors in column:

assuming direct connection

How many procs in each column?

What is the message size per proc?

What is the communication time?

- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu\, m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu\, m\, p)$

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$
  - Broadcast vector elements to processors in column: $O\left(\tau \log \sqrt{n} + \mu \frac{n}{\sqrt{p}} \log \sqrt{n}\right)$

assuming direct connection

Each column has $\sqrt{p}$ procs

message size $O(n/\sqrt{p})$

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$
  - Broadcast vector elements to processors in column: $O\left(\tau \log \sqrt{n} + \mu \frac{n}{\sqrt{p}} \log \sqrt{n}\right)$
  - Local mult

assuming direct connection

What is the computation time?

How much of the matrix is on each processor?

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$
  - Broadcast vector elements to processors in column:
  - Local multiplication: $O\left($
  - Sum partial y values in

assuming direct connection

What is the communication time?

What is the message size per proc?

How many procs in each row?

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$
  - Broadcast vector elements to processors in column: $O\left(\tau \log \sqrt{p} + \mu \frac{n}{\sqrt{p}} \log \sqrt{p}\right)$
  - Local multiplication: $O\left(\frac{n^2}{p}\right)$
  - Sum partial y values in each row: $O\left(\tau \log \sqrt{p} + \mu \frac{n}{\sqrt{p}} \log \sqrt{p}\right)$

assuming direct connection

Each row has $\sqrt{p}$ procs

message size $O(n/\sqrt{p})$

2D Partitioning for Matrix-Vector Multiply

$p < n^2$

- Algorithm
  - align vector along the main diagonal: $O\left(\tau + \mu \cdot \frac{n}{\sqrt{p}}\right)$
  - Broadcast vector elements to processors in column: $O\left(\tau \log \sqrt{p} + \mu \frac{n}{\sqrt{p}} \log \sqrt{p}\right)$
  - Local multiplication: $O\left(\frac{n^2}{p}\right)$
  - Sum partial y values in each row: $O\left(\tau \log \sqrt{p} + \mu \frac{n}{\sqrt{p}} \log \sqrt{p}\right)$
- Communication time = $O\left(\tau \log \sqrt{p} + \mu \frac{n}{\sqrt{p}} \log \sqrt{p}\right)$
- Computation time = $O\left(\frac{n^2}{p}\right)$

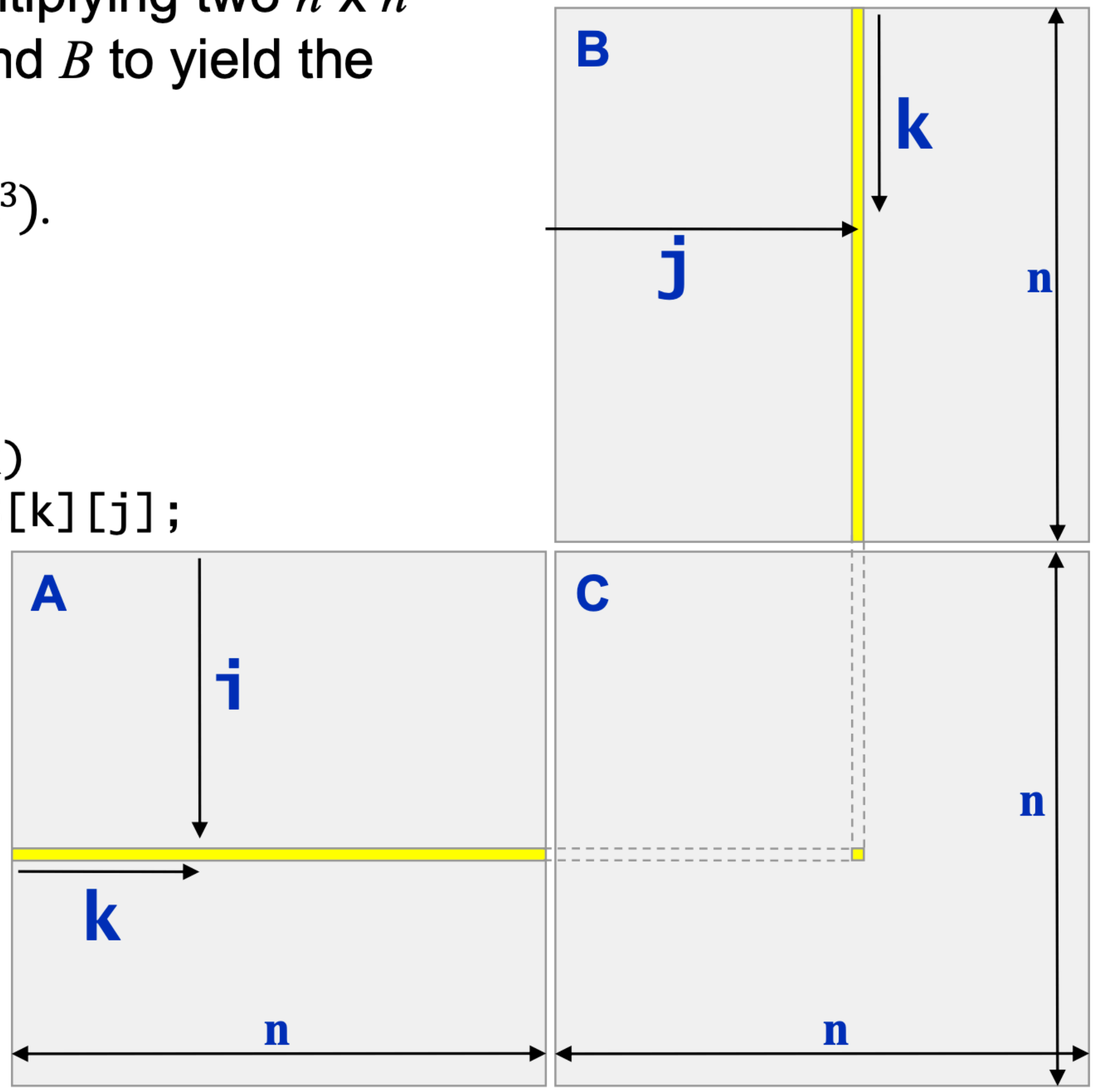
assuming direct connection

Matrix-Matrix Multiplication

Matrix-Matrix Multiplication

- Consider the problem of multiplying two n x n dense, square matrices A and B to yield the product matrix $C = A$ x B.
- The serial complexity is $O(n^3)$.

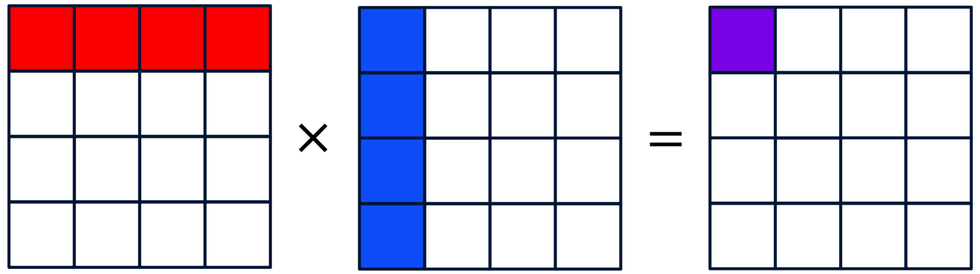
```
for (i = 0; i < n; ++i)
    for (j = 0; j < n; ++j) {
        double sum = 0;
        for (k = 0; k < n; ++k)
            sum += A[i][k] * B[k][j];
        C[i][j] = sum;
    }
}
```



Initial Algorithm for Matrix-Matrix Multiplication

Assume we have n^2 processors, where every processor (i, j) has elements A[i, j] and B[i, j] in the initial distribution.

For processor (i, j) to compute C[i, j], it needs the i-th row of A and j-th col of B.

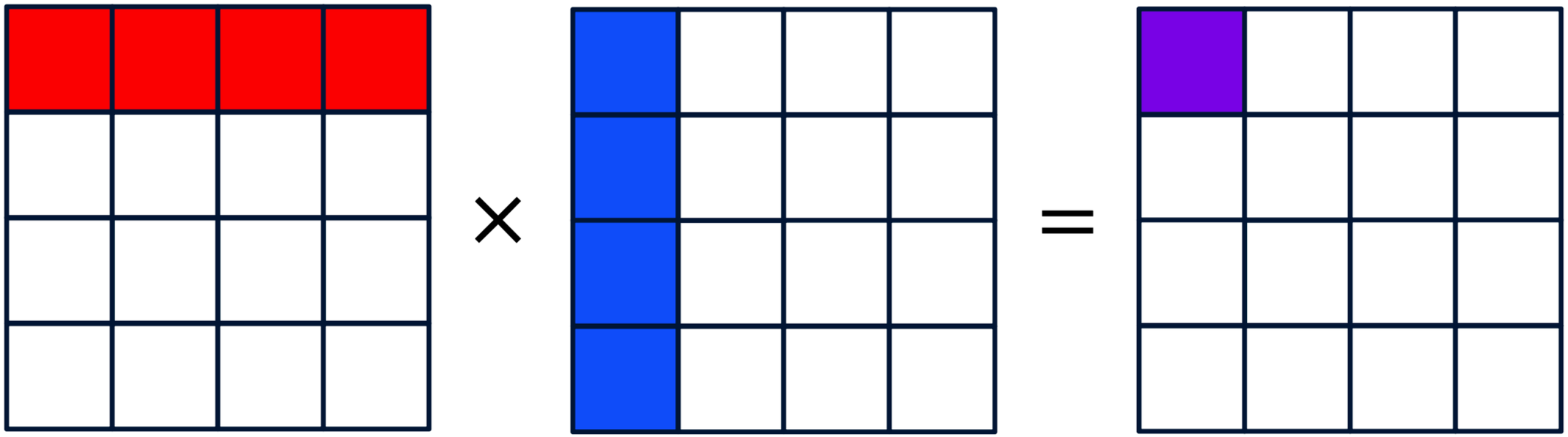


Idea:

- Allgather i-th row of A onto each processor for each row i
- Allgather j-th col of B onto each processor for each col j
- Perform dot product of row and col to get C[i, j]

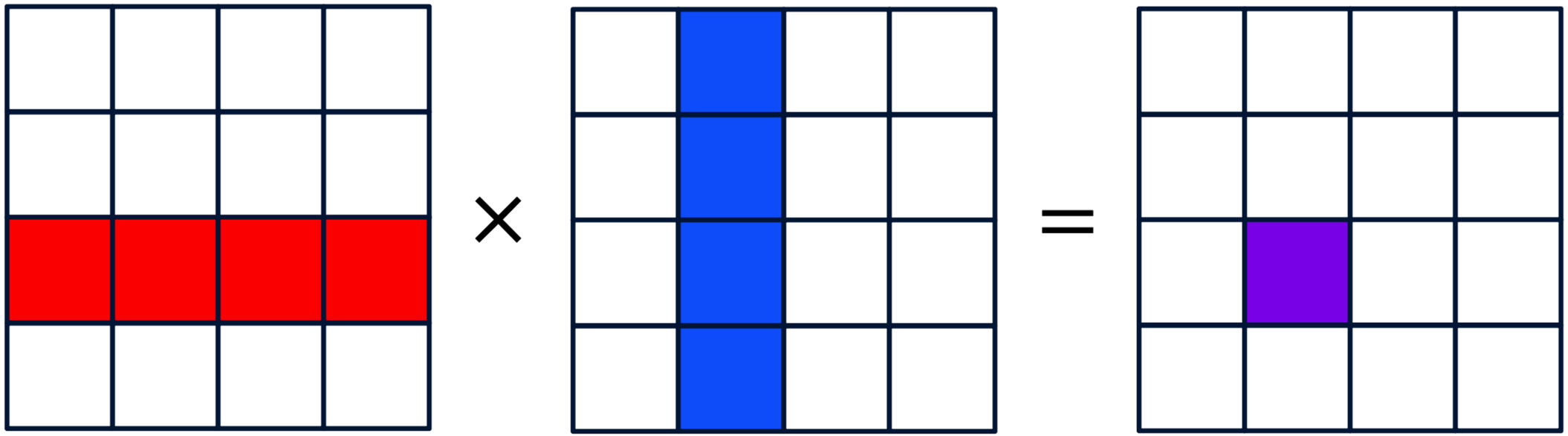
Data Requirements for Matrix-Matrix Multiply

Consider data needed for output matrix block shown in purple



Data Requirements for Matrix-Matrix Multiply

Consider data needed for output matrix block shown in purple



Matrix-Matrix Multiplication

- Consider the problem of multiplying two $n \times n$ dense, square matrices A and B to yield the product matrix $C = A \times B$.

rial complexity is $O(n^3)$.

- The se

› not consider better serial algorithms (Strassen's method)

- We do

these can still be used as serial kernels in the parallel algorithms.

Note:

Matrix-Matrix Multiplication

- Consider the problem of multiplying two $n \times n$ dense, square matrices A and B to yield the product matrix $C = A \times B$.

rial complexity is $O(n^3)$.

- The se
  - We do not consider better serial algorithms (Strassen's method)

these can still be used as serial kernels in the parallel algorithms.

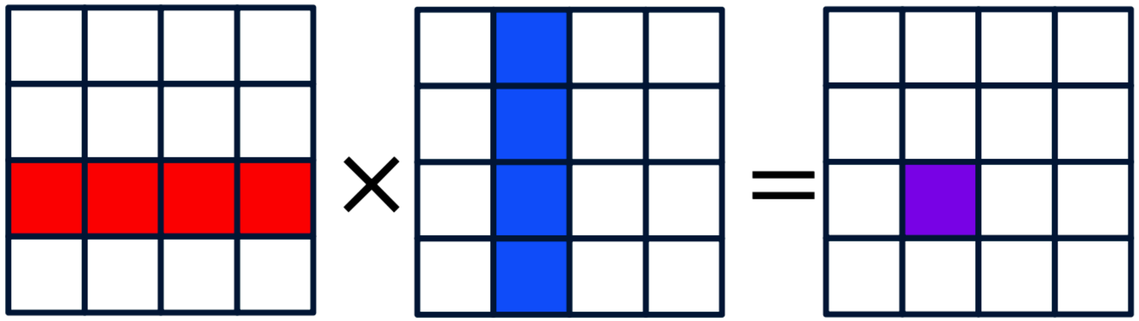
Note:

- A useful concept in this case is called *block* operations
  - an $n \times n$ matrix A can be regarded as a $q \times q$ array of blocks $A_{i,j}$ $(0 \leq i, j < q)$ such that each block is an $(n/q) \times (n/q)$ submatrix.

- In this view, we perform q^3 matrix multiplications, each involving $(n/q) \times (n/q)$ matrices.

Blocked Matrix-Matrix Multiply

- Consider two $n \times n$ matrices A and B partitioned into p blocks $A_{i,j}$ and $B_{i,j}$ $(0 \leq i, j < \sqrt{p})$ of size $(n/\sqrt{p}) \times (n/\sqrt{p})$ each.
- Process $P_{i,j}$ initially stores $A_{i,j}$ and $B_{i,j}$ and computes block $C_{i,j}$ of the result matrix.
- Computing submatrix $C_{i,j}$ requires all submatrices $A_{i,k}$ and $B_{k,j}$ for $0 \leq k < \sqrt{p}$.
  - AllGather blocks of A along rows and B along columns.
  - Perform local submatrix multiplication.



Analysis of Blocked Matrix-Matrix Multiply

- The two AllGather operations take time:

What is the communication time?

What is the message size per proc? How many procs participate in each allgather?

- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu m p)$

Analysis of Blocked Matrix-Matrix Multiply

- The two AllGather operations take time:
  - $O(\tau \log \sqrt{p} + \mu \frac{n^2}{p} \sqrt{p})$

- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu\, m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu\, m\, p)$

Analysis of Blocked Matrix-Matrix Multiply

- The two AllGather operations take time:
  - $O(\tau \log \sqrt{p} + \mu \frac{n^2}{p} \sqrt{p})$
- The computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{p}) \times (n/\sqrt{p})$ sized submatrices:

What is the computation time? $\left(\frac{n^3}{p}\right)$

Analysis of Blocked Matrix-Matrix Multiply

- The two AllGather operations take time:
  - $O(\tau \log \sqrt{p} + \mu \frac{n^2}{p} \sqrt{p})$
- The computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{p}) \times (n/\sqrt{p})$ sized submatrices:
  - $O\left(\sqrt{p} \times \left(\frac{n}{\sqrt{p}}\right)^3\right) = O\left(\frac{n^3}{p}\right)$
- Total Time: $O\left(\frac{n^3}{p} + \tau \log p + \mu \frac{n^2}{\sqrt{p}}\right)$

Analysis of Blocked Matrix-Matrix Multiply

- The two AllGather operations take time:
  - $O(\tau \log \sqrt{p} + \mu \frac{n^2}{p} \sqrt{p})$
- The computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{n}) \times (n/\sqrt{p})$ sized submatrices:
  - $O\left(\sqrt{p} \times \left(\frac{n}{\sqrt{p}}\right)^3\right) = O\left(\frac{n^3}{p}\right)$
- Total Time: $O\left(\frac{n^3}{p} + \tau \log p + \mu \frac{n^2}{\sqrt{p}}\right)$

Any issues with this algorithm?

Analysis of Blocked Matrix-Matrix Multiply

- The two AllGather operations take time:
  - $O(\tau \log \sqrt{p} + \mu \frac{n^2}{p} \sqrt{p})$
- The computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{n}) \times$ $(n/\sqrt{p})$ sized submatrices:
  - $O\left(\sqrt{p} \times \left(\frac{n}{\sqrt{p}}\right)^3\right) = O\left(\frac{n^3}{p}\right)$
- Total Time: $O\left(\frac{n^3}{p} + \tau \log p + \mu \frac{n^2}{\sqrt{p}}\right)$

t for $p = O(n^2)$

not memory

- Major drawback of the algorithm is that it is n optimal.

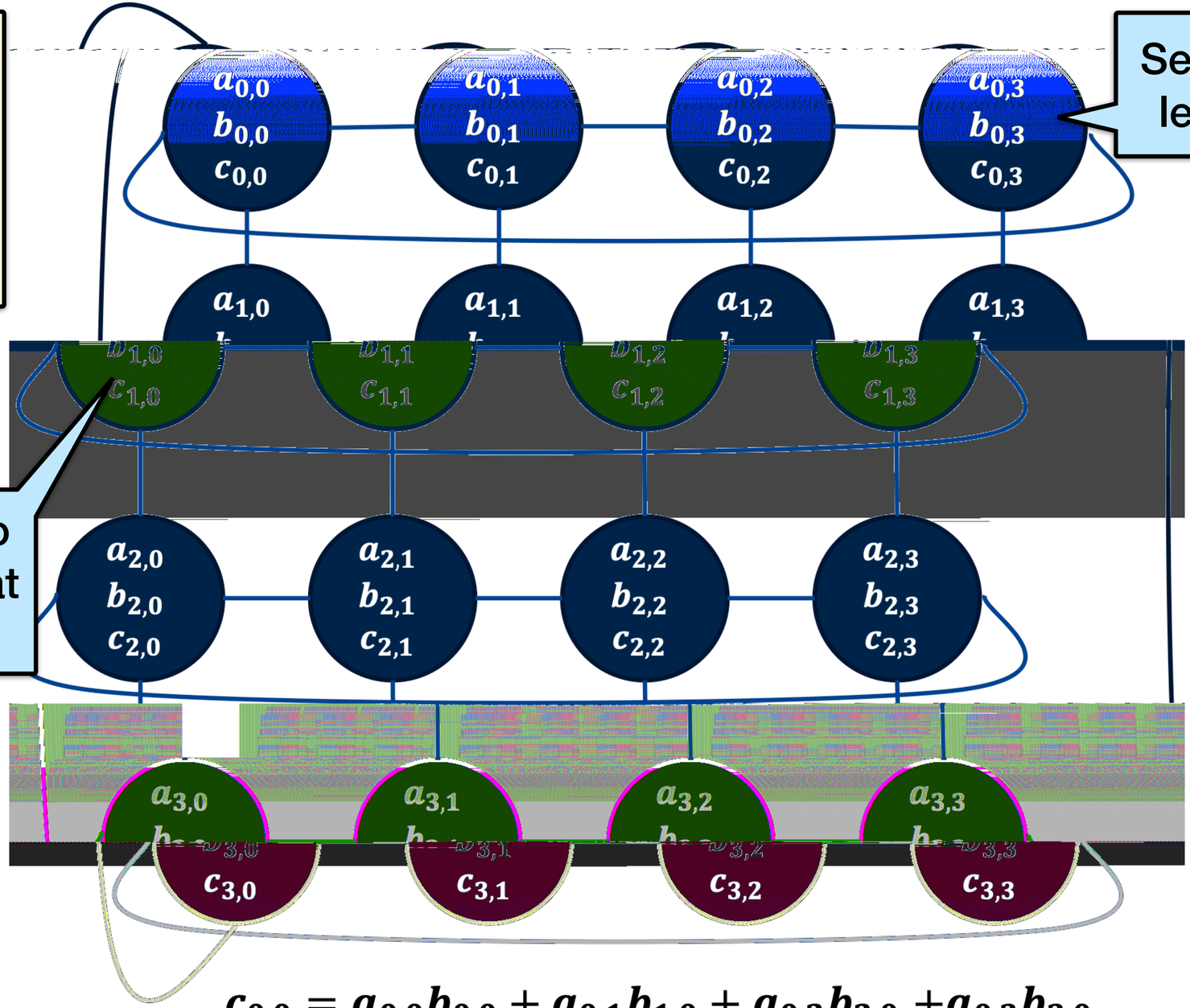
Extra $\sqrt{p}$ factor of memory consumption!

Ideal: Memory is serial_memory / p

Restriction - one instance of each element in A, B across all processors

Send a elements to the left one hop at a time

Send b elements to the north one hop at a time

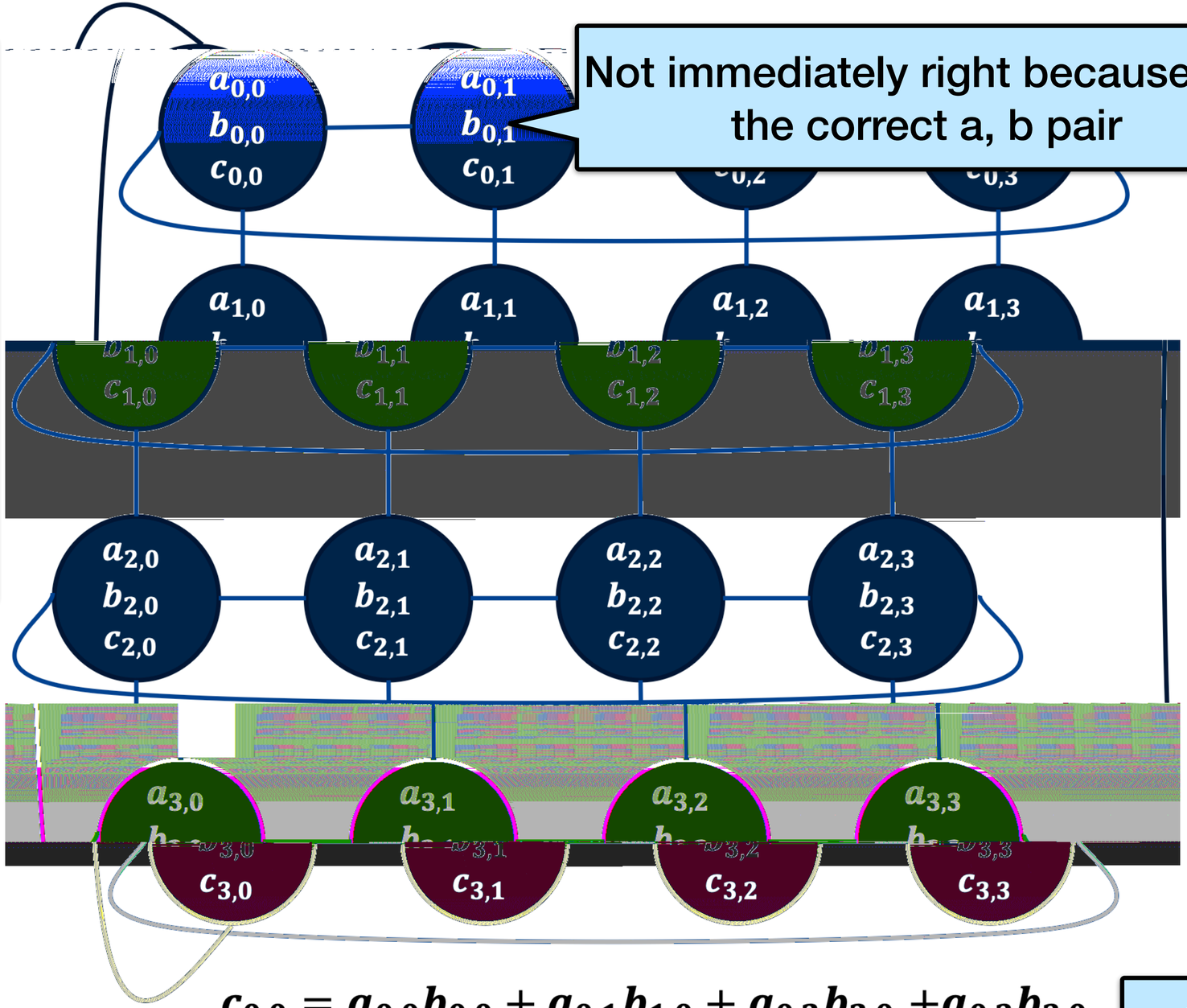


$$c_{0,0} = a_{0,0}b_{0,0} + a_{0,1}b_{1,0} + a_{0,2}b_{2,0} + a_{0,3}b_{3,0}$$

$$c_{0,1} = a_{0,0}b_{0,1} + a_{0,1}b_{1,1} + a_{0,2}b_{2,1} + a_{0,3}b_{3,1}$$

Goal - all processors are computing in each round

We need the corresponding a and b on each processor in each round



Not immediately right because not the correct a, b pair

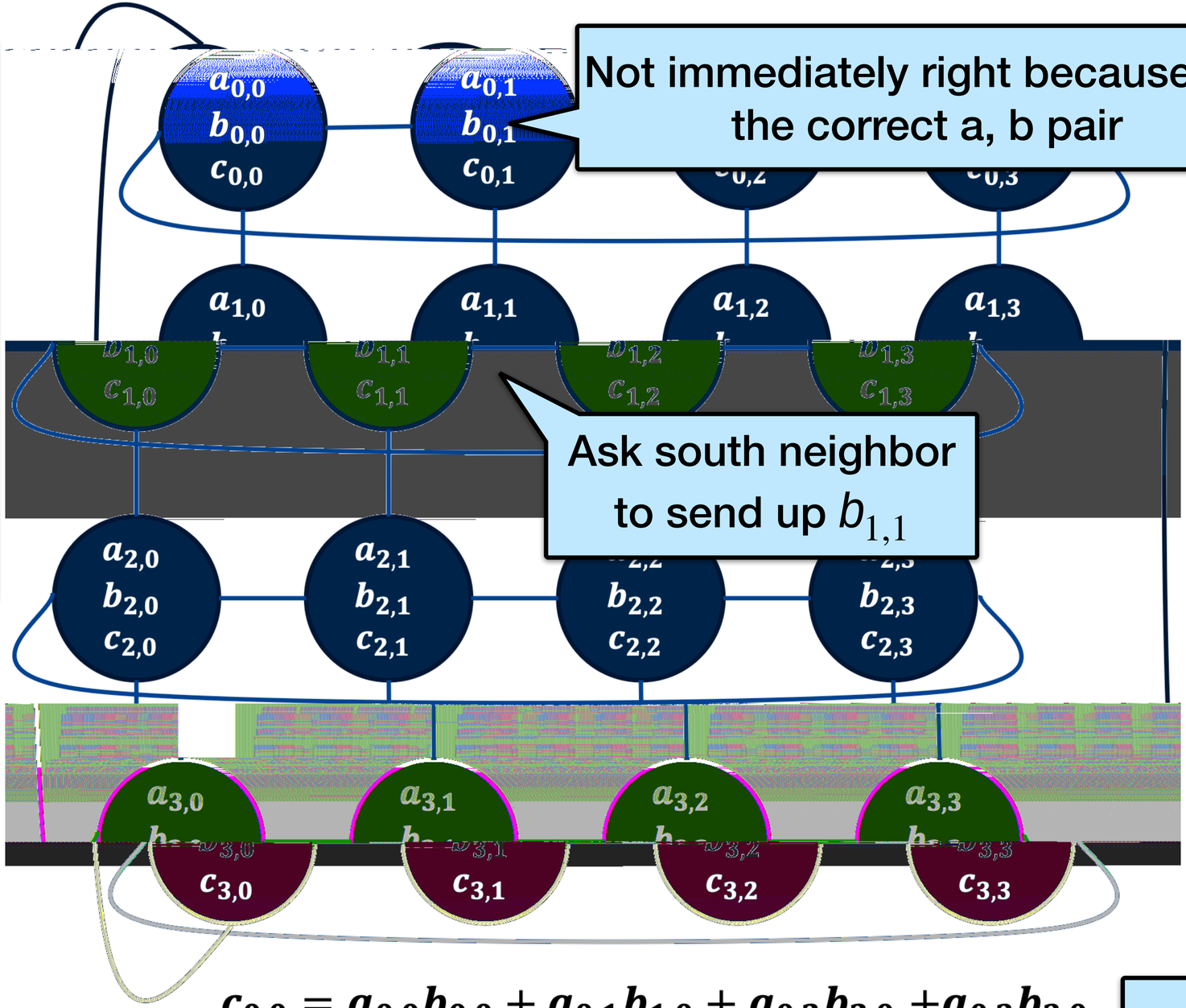
$$c_{0,0} = a_{0,0}b_{0,0} + a_{0,1}b_{1,0} + a_{0,2}b_{2,0} + a_{0,3}b_{3,0}$$

$$c_{0,1} = a_{0,0}b_{0,1} + a_{0,1}b_{1,1} + a_{0,2}b_{2,1} + a_{0,3}b_{3,1}$$

Can compute the terms in any order

Goal - all processors are computing in each round

We need the corresponding a and b on each processor in each round



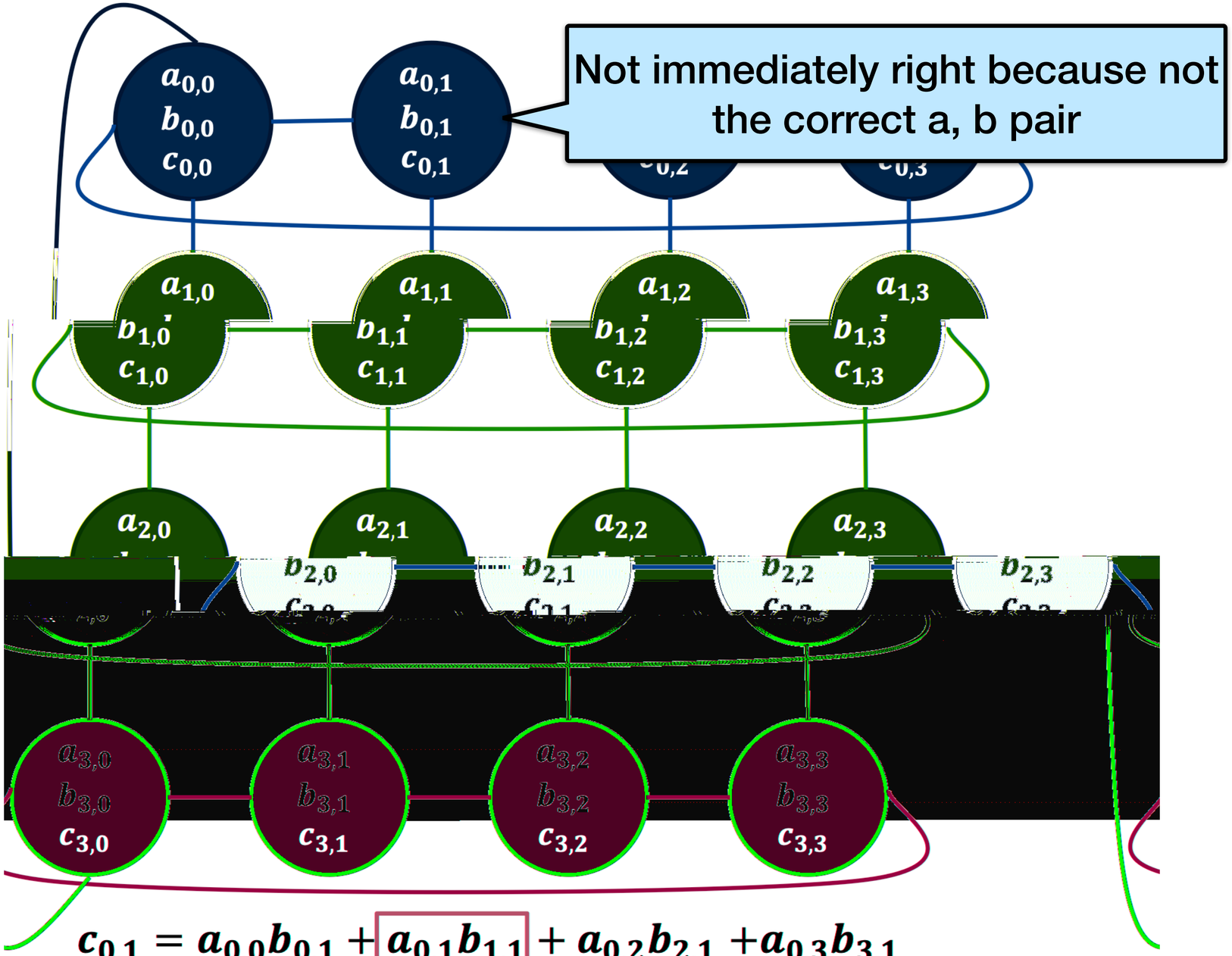
Not immediately right because not the correct a, b pair

Ask south neighbor to send up $b_{1,1}$

$$c_{0,0} = a_{0,0}b_{0,0} + a_{0,1}b_{1,0} + a_{0,2}b_{2,0} + a_{0,3}b_{3,0}$$

$$c_{0,1} = a_{0,0}b_{0,1} + a_{0,1}b_{1,1} + a_{0,2}b_{2,1} + a_{0,3}b_{3,1}$$

Can compute the terms in any order

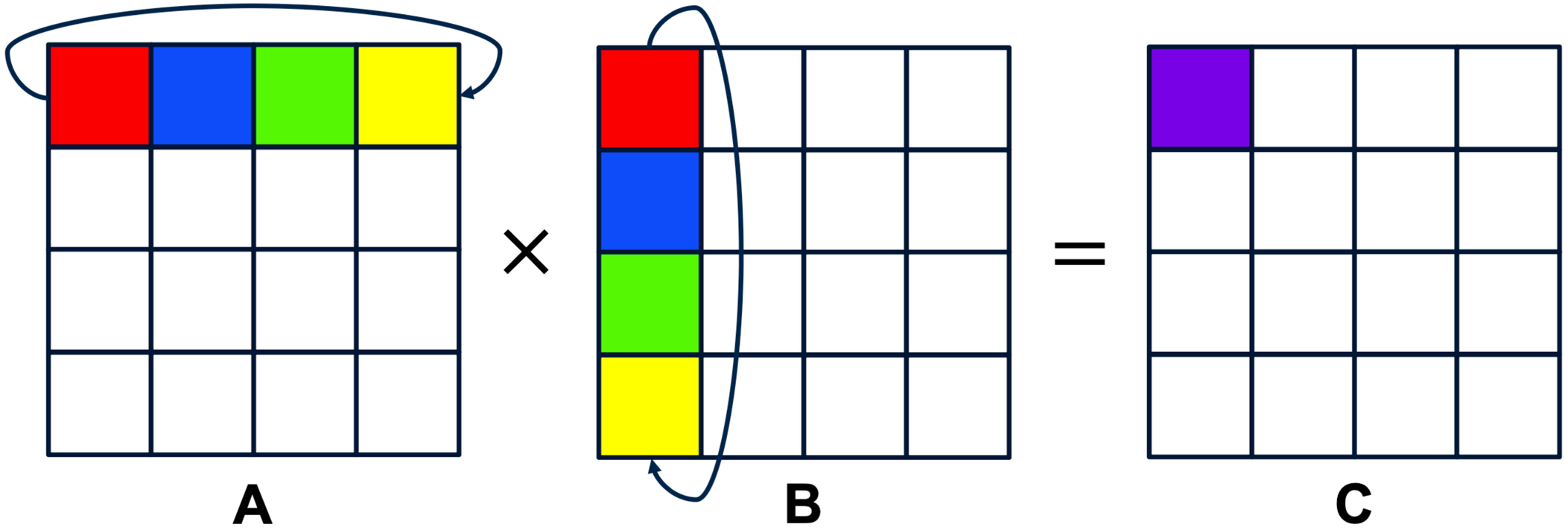


$$c_{0,1} = a_{0,0}b_{0,1} + \boxed{a_{0,1}b_{1,1}} + a_{0,2}b_{2,1} + a_{0,3}b_{3,1}$$

$$c_{0,2} = a_{0,0}b_{0,2} + a_{0,1}b_{1,2} + \boxed{a_{0,2}b_{2,2}} + a_{0,3}b_{3,2}$$

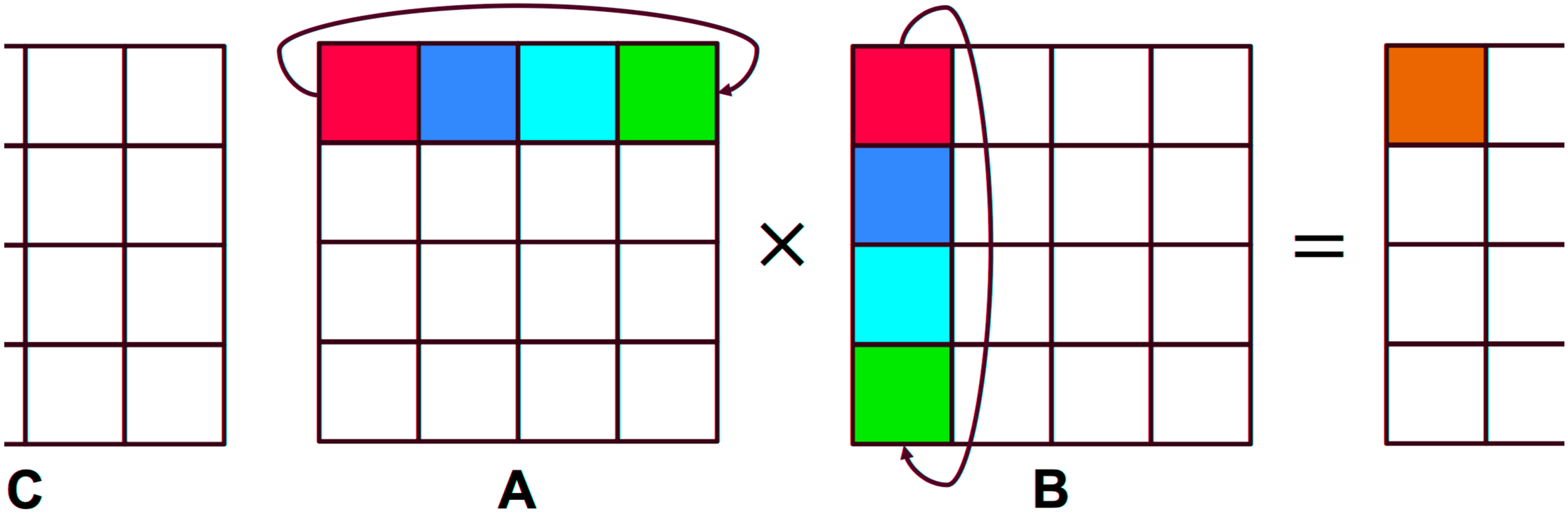
Cannon's Algorithm: Rotations

Consider processor $P_{0,0}$: Step 1 - Multiply the Red blocks



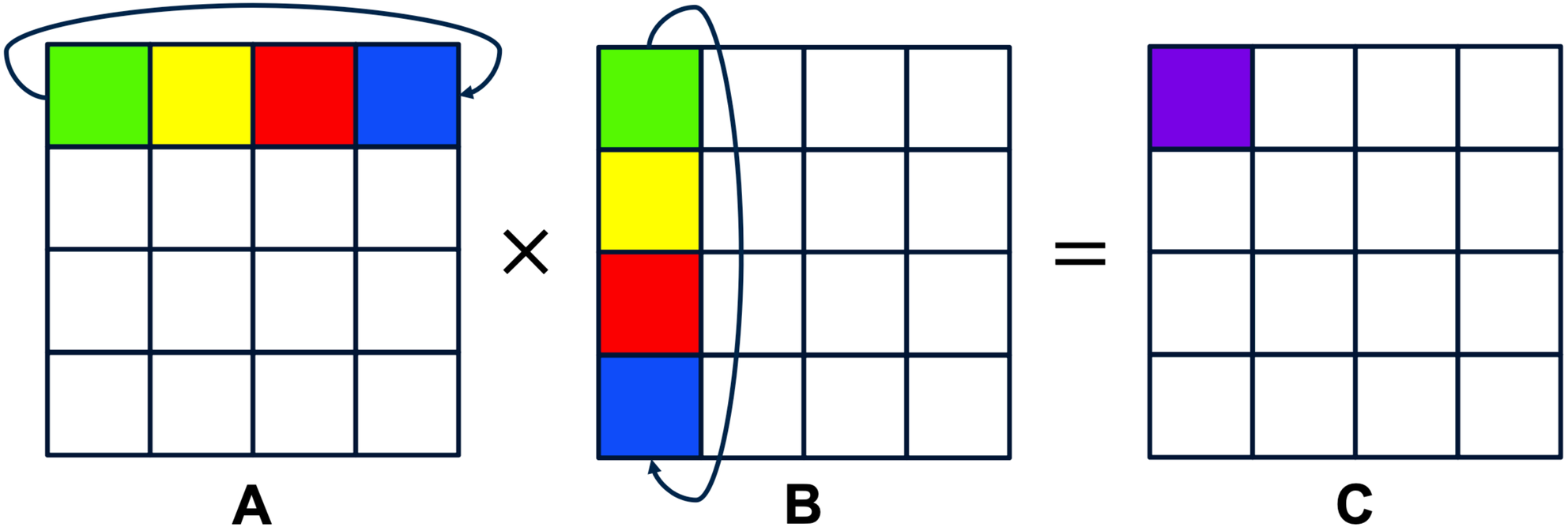
Cannon's Algorithm: Rotations

$P_{0,0}$: Step 2 – Left shift A, Up shift B, Multiply the Blue blocks



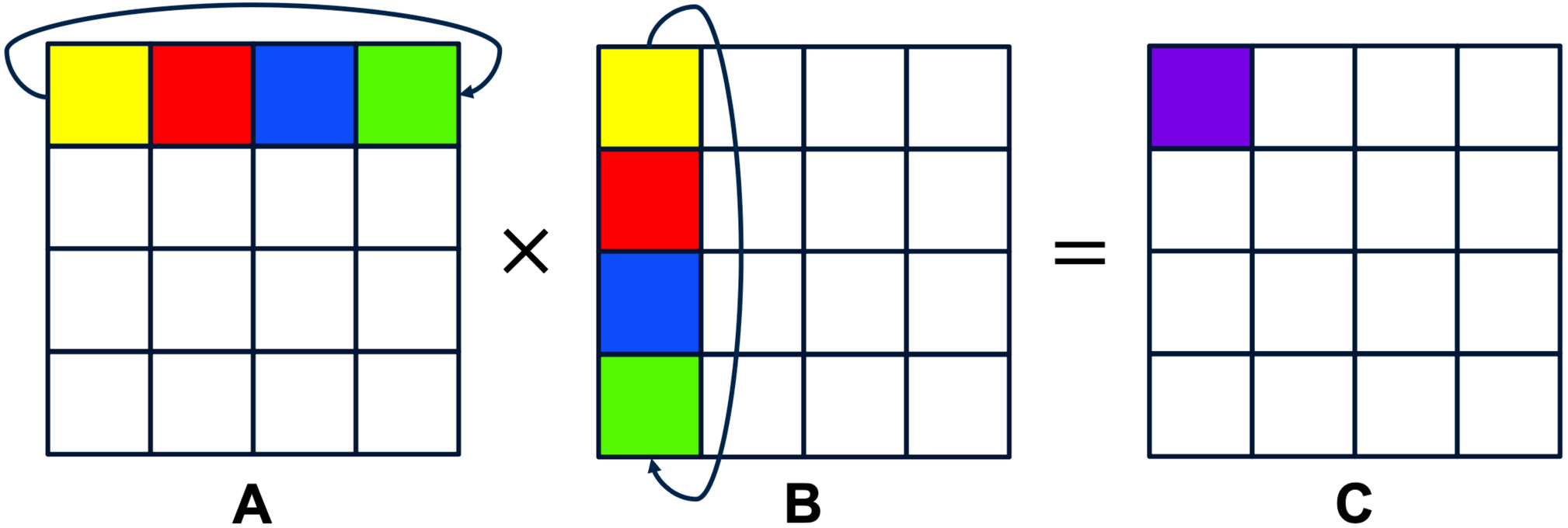
Cannon's Algorithm: Rotations

$P_{0,0}$: Step 3 – Left shift A, Up shift B, Multiply the Green blocks



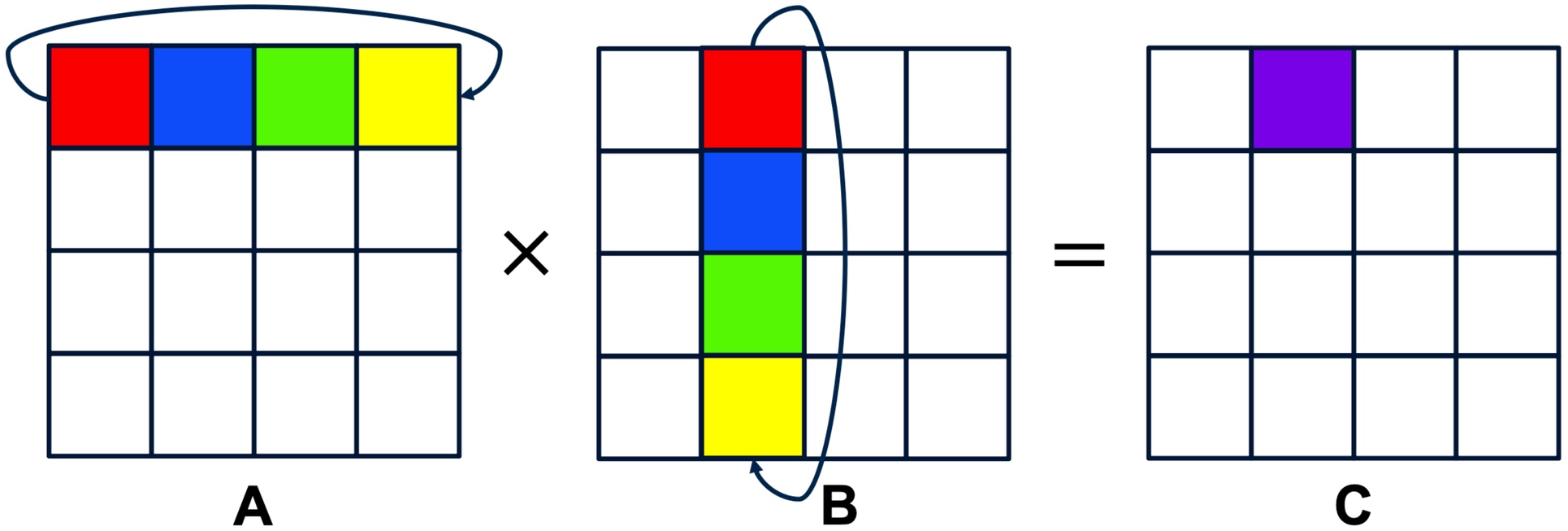
Cannon's Algorithm: Rotations

$P_{0,0}$: Step 4 – Left shift A, Up shift B, Multiply the Yellow blocks

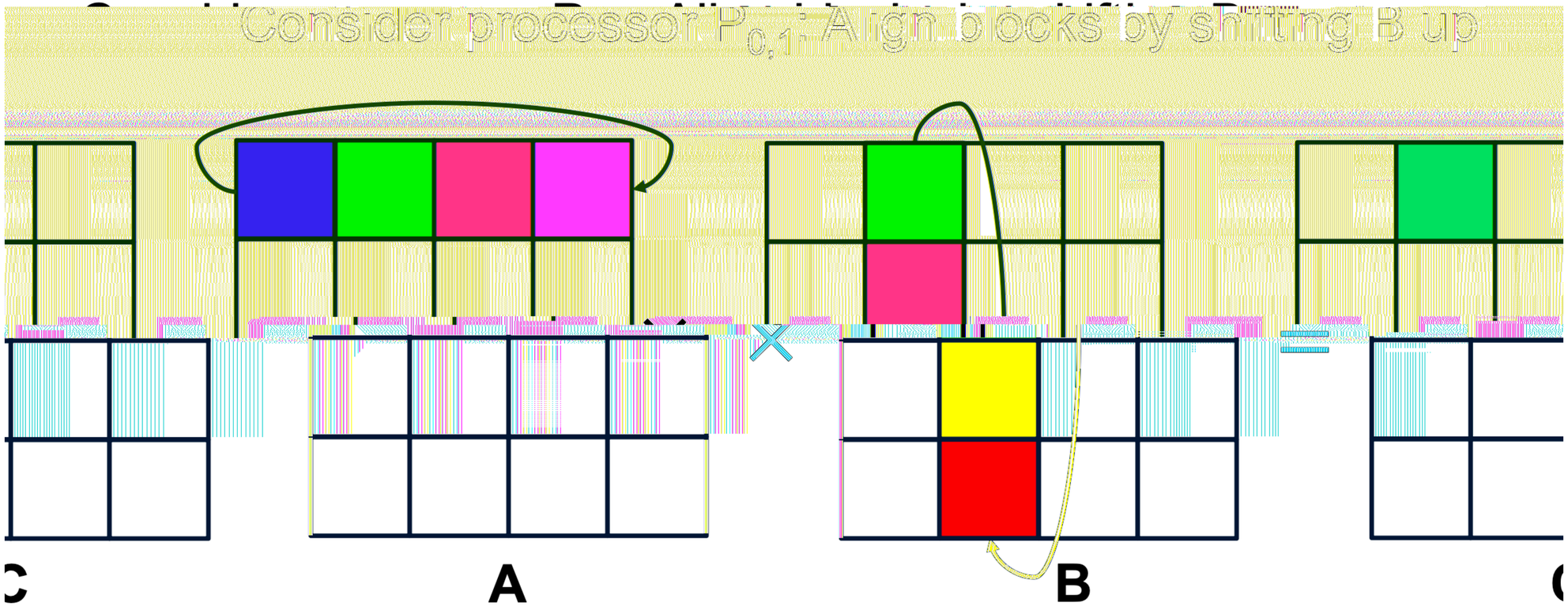


Cannon's Algorithm: Initialization

Consider processor $P_{0,1}$

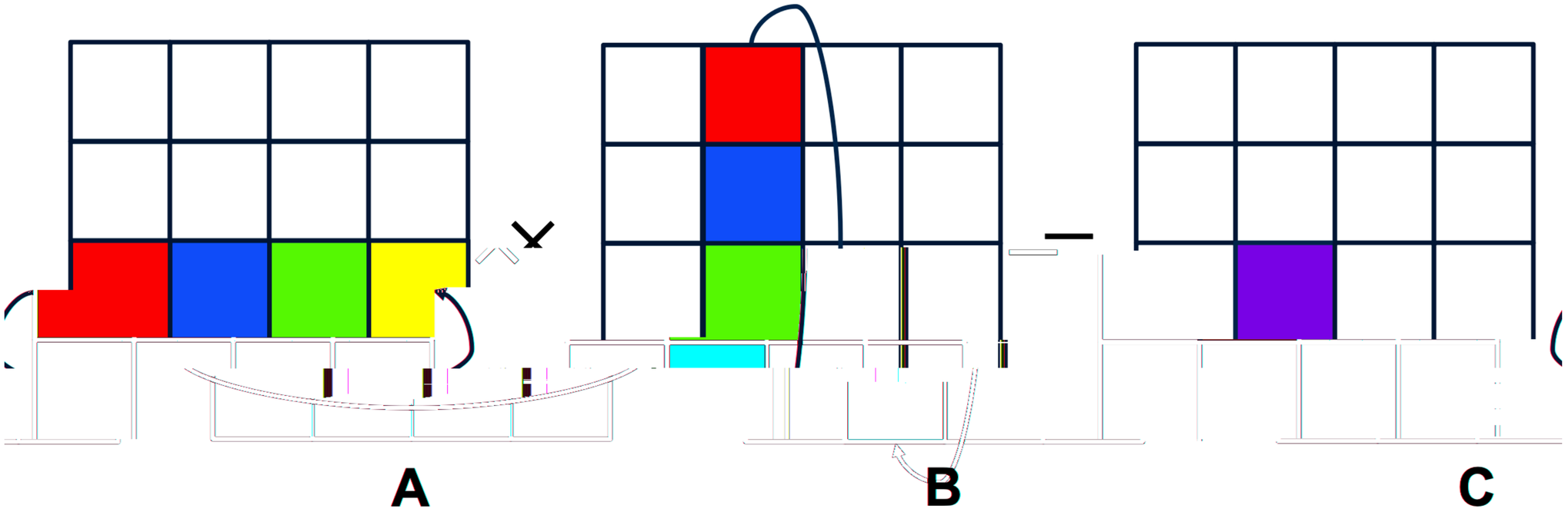


Cannon's Algorithm: Initialization



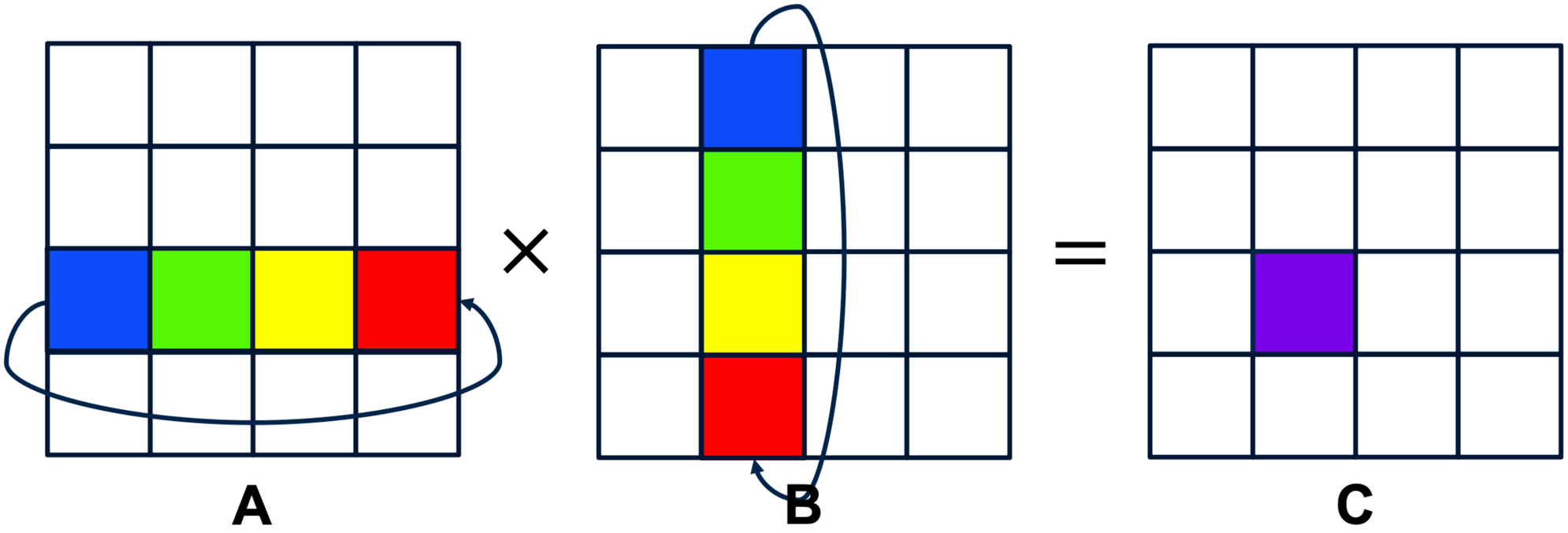
Cannon's Algorithm

- Consider processor $P_{2,1}$
- We need to align the blocks!
  - In row 2 shift A_{ij} left by 2
  - In column 1 shift B_{ij} up by 1



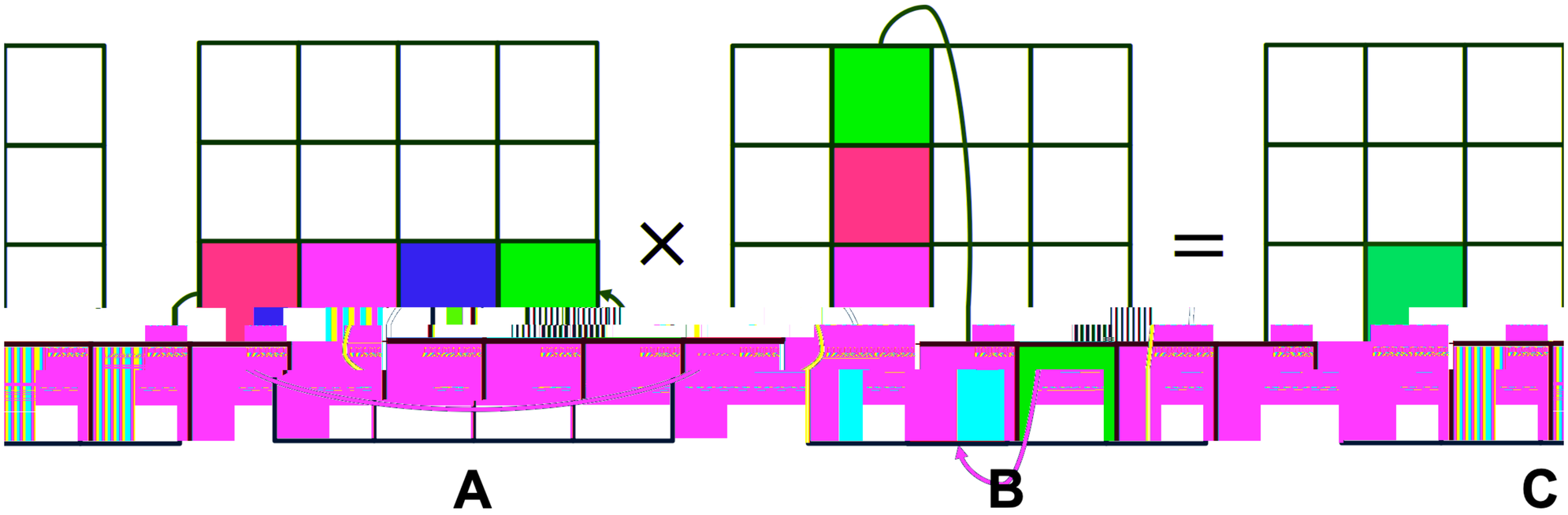
Cannon's Algorithm

- Consider processor $P_{2,1}$
- Alignment Step 1



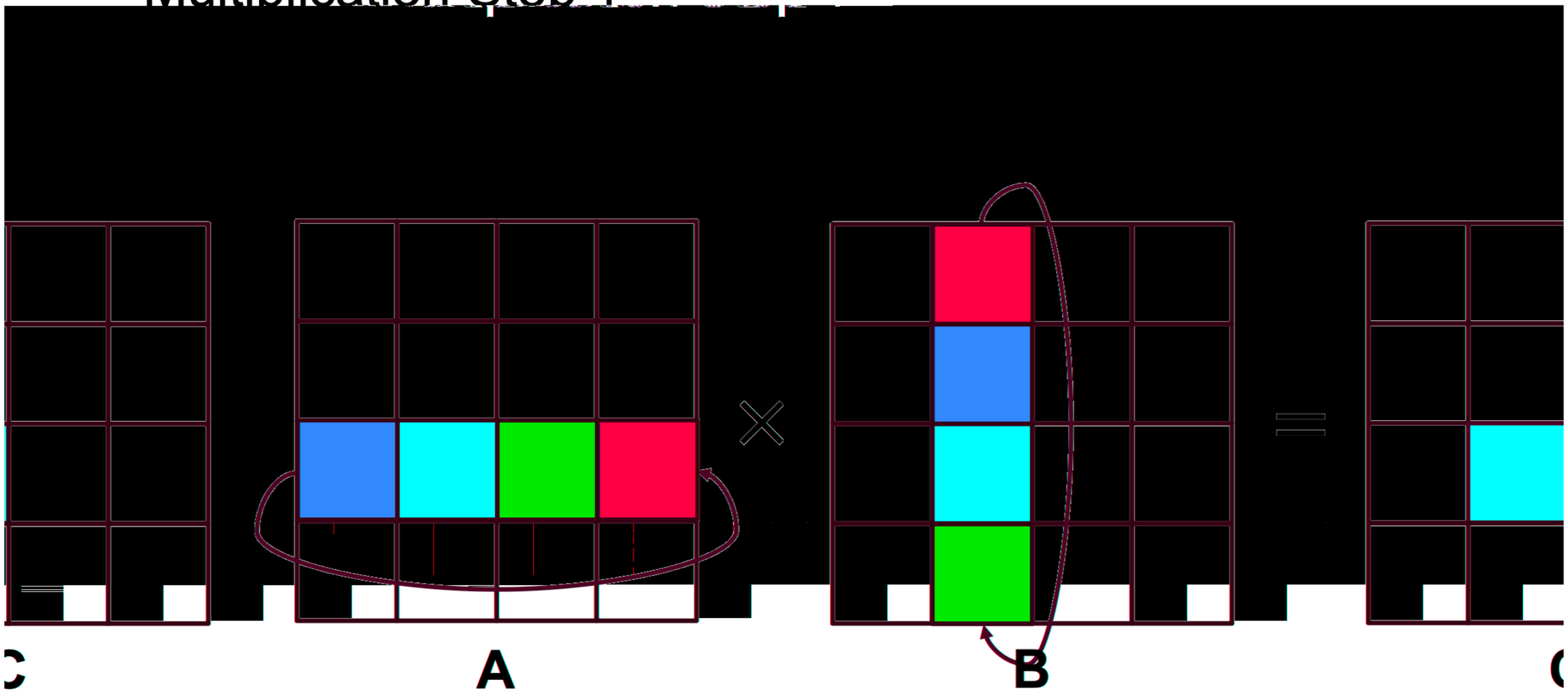
Cannon's Algorithm

- Consider processor $P_{2,1}$
- Alignment Step 2



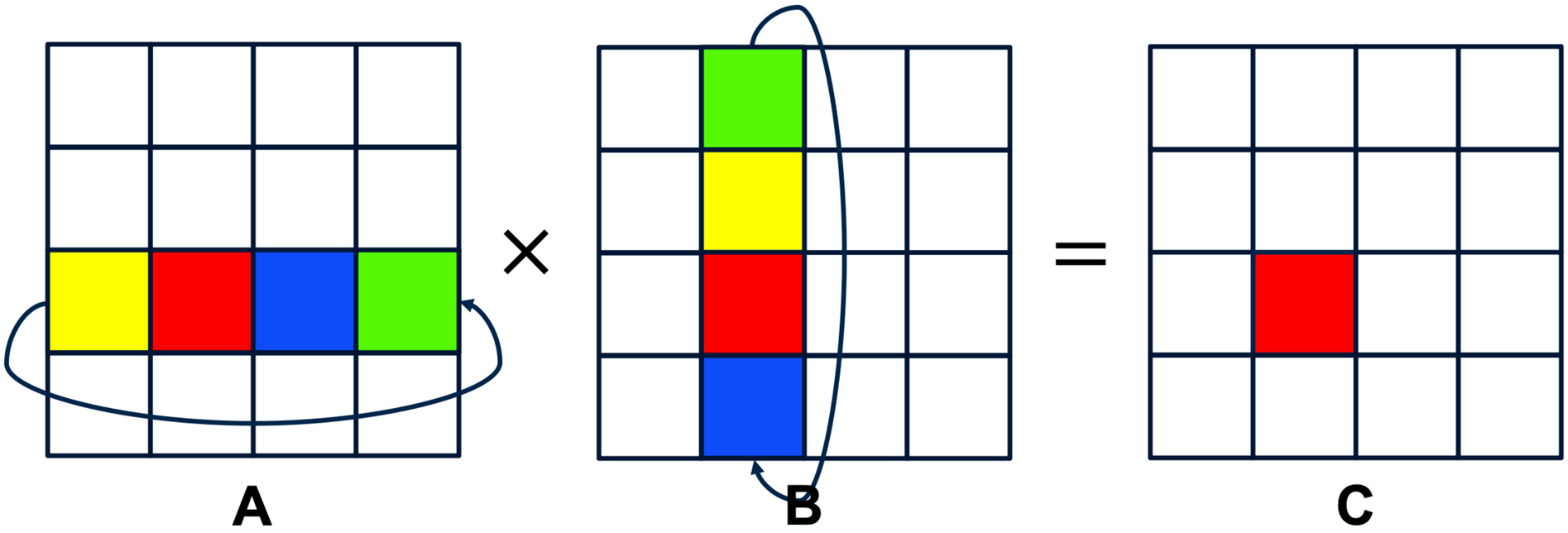
Cannon's Algorithm

- Consider processor $P_{2,1}$
- Multiplication Step 1



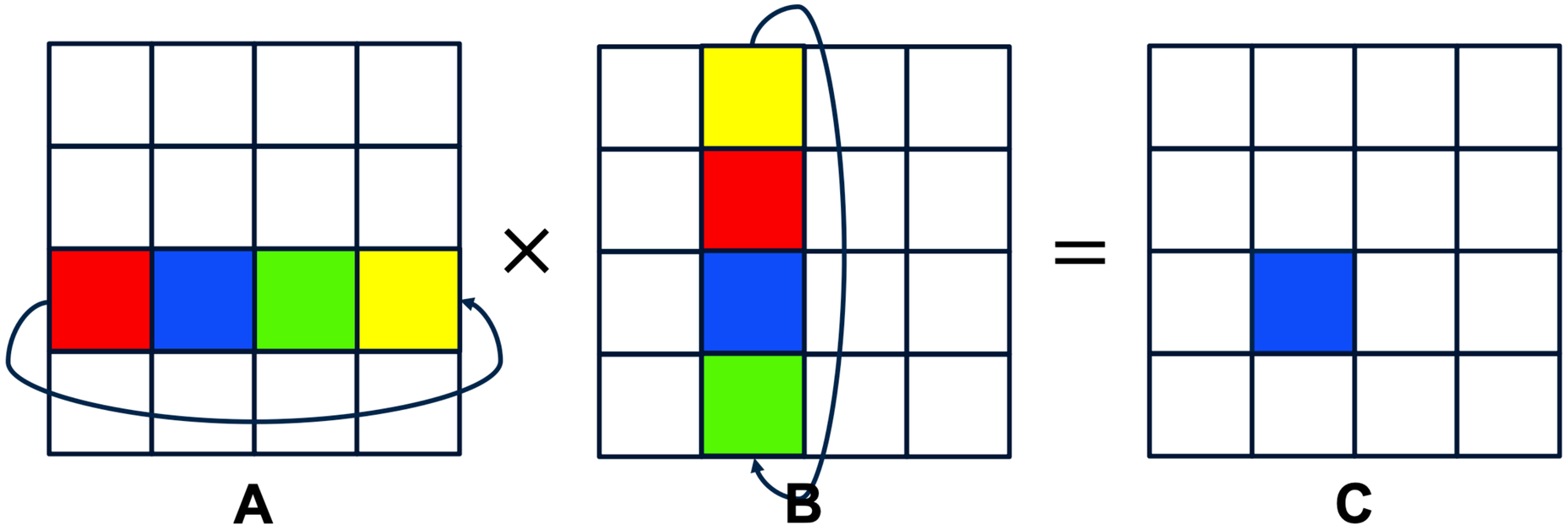
Cannon's Algorithm

- Consider processor $P_{2,1}$
- Multiplication Step 2



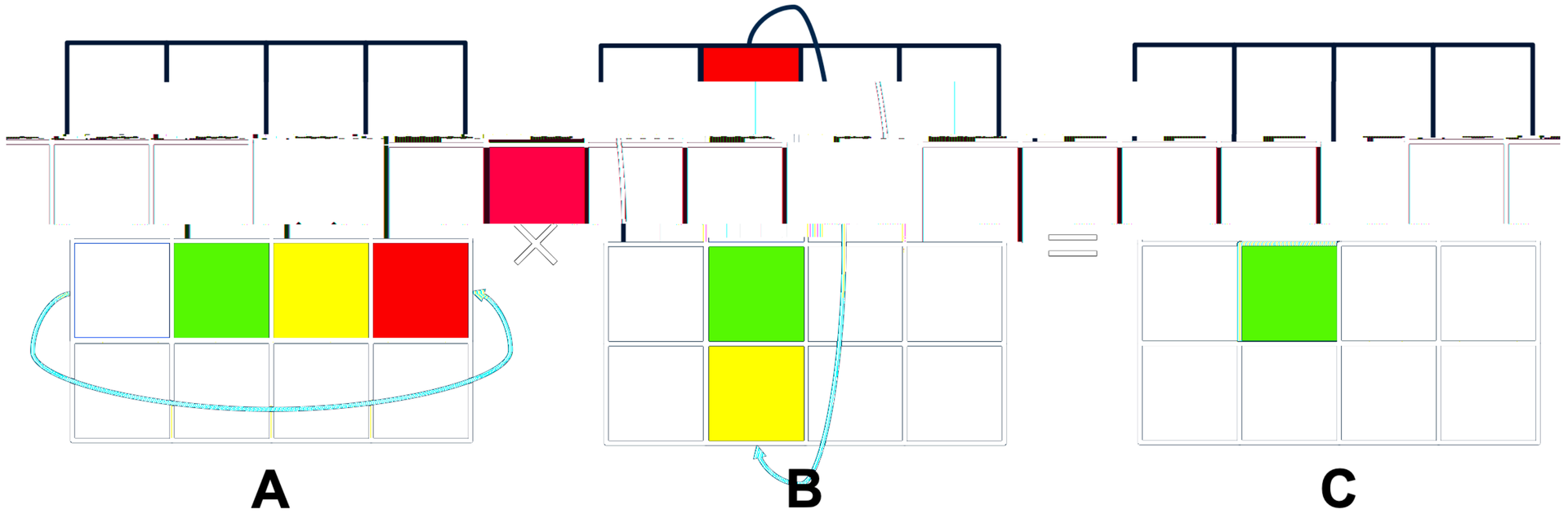
Cannon's Algorithm

- Consider processor $P_{2,1}$
- Multiplication Step 3

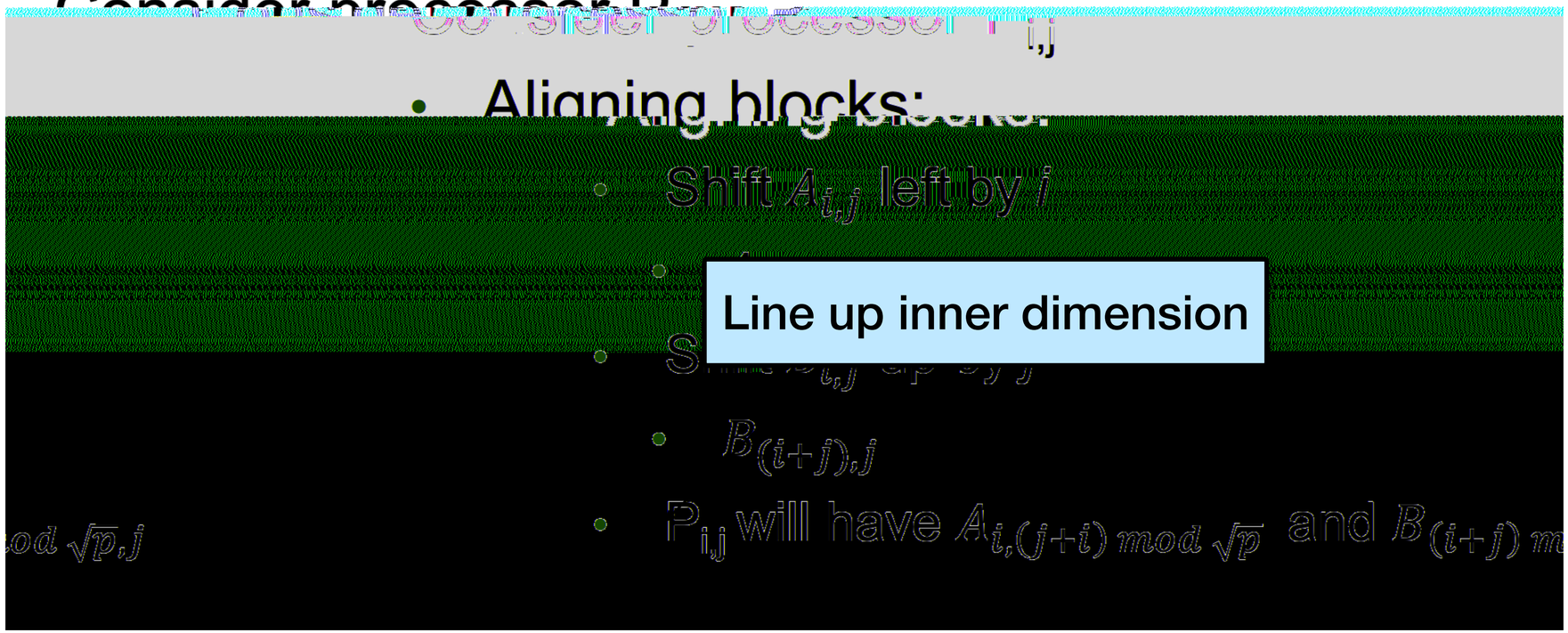


Cannon's Algorithm

- Consider processor $P_{2,1}$
- Multiplication Step 4



Cannon's Algorithm: Initial Alignment



Analysis of Cannon's Algorithm

- In the alignment step,
  - maximum distance over which a block shifts is $\sqrt{p} - 1$,
  - two shift operations require a total of $O\left(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p}\right)$ time.

Analysis of Cannon's Algorithm

- In the alignment step,
  - maximum distance over which a block shifts is $\sqrt{p} - 1$,
  - two shift operations require a total of $O\left(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p}\right)$ time.
- Compute-and-shift phase has $\sqrt{p}$ steps
  - Computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{p}) \times (n/\sqrt{p}$ submatrices: $O\left(\sqrt{p} \times \left(\frac{n}{\sqrt{p}}\right)^3\right) = O\left(\frac{n^3}{p}\right)$
  - Communication requires $\sqrt{p}$ shifts $O(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p})$

) sized

Analysis of Cannon's Algorithm

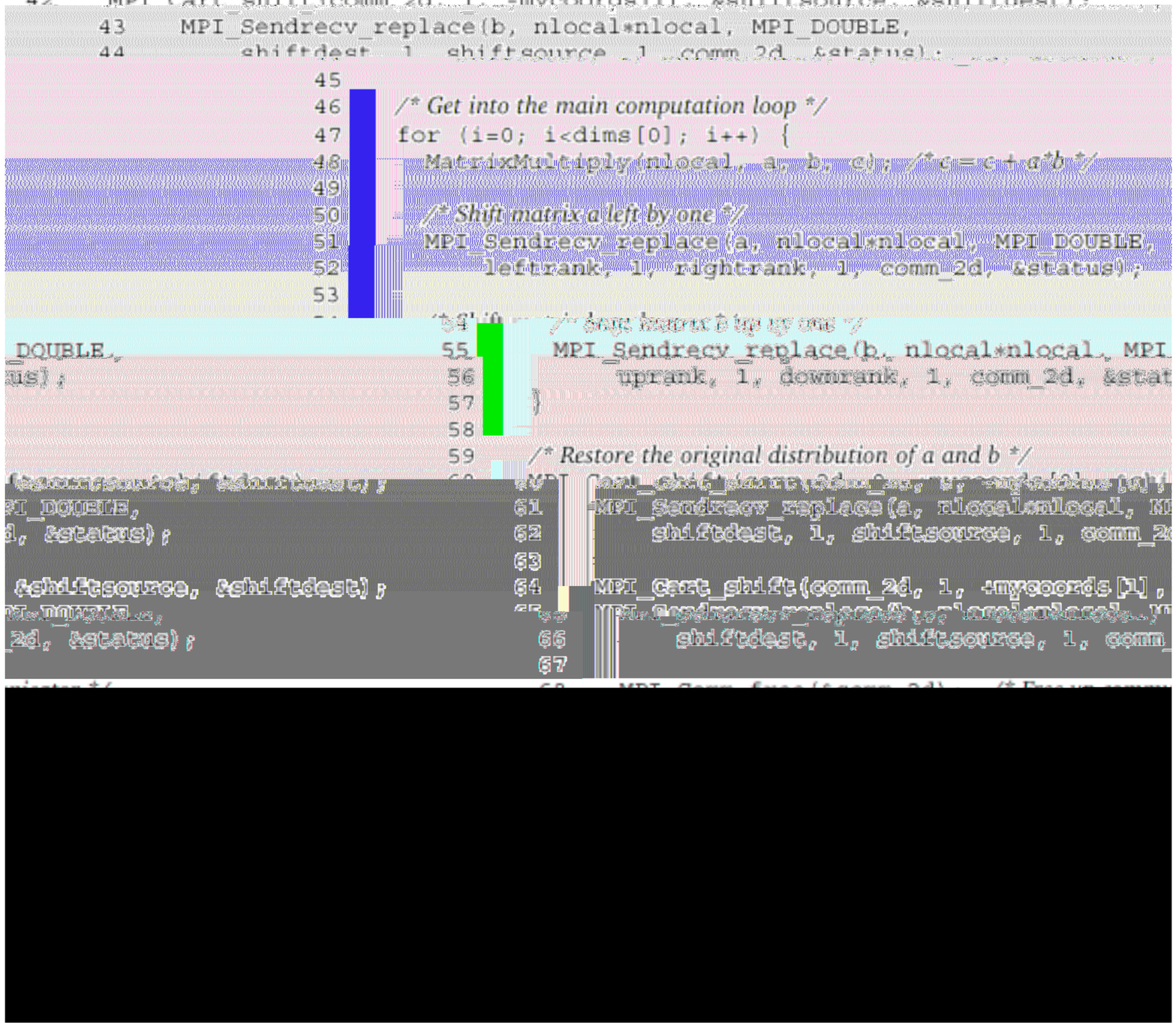
- In the alignment step,
  - maximum distance over which a block shifts is $\sqrt{p} - 1$,
  - two shift operations require a total of $O\left(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p}\right)$ time.
- Compute-and-shift phase has $\sqrt{p}$ steps
  - Computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{p}) \times (n/\sqrt{p}$ submatrices: $O\left(\sqrt{p} \times \left(\frac{n}{\sqrt{p}}\right)^3\right) = O\left(\frac{n^3}{p}\right)$
  - Communication requires $\sqrt{p}$ shifts $O(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p})$
- Total Time: $O\left(\frac{n^3}{p} + \tau\sqrt{p} + \mu\frac{n^2}{\sqrt{p}}\right)$

Analysis of Cannon's Algorithm

- In the alignment step,
  - maximum distance over which a block shifts is $\sqrt{p} - 1$,
  - two shift operations require a total of $O\left(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p}\right)$ time.
- Compute-and-shift phase has $\sqrt{p}$ steps
  - Computation requires $\sqrt{p}$ multiplications of $(n/\sqrt{p}) \times (n/\sqrt{p}$ submatrices: $O\left(\sqrt{p} \times \left(\frac{n}{\sqrt{p}}\right)^3\right) = O\left(\frac{n^3}{p}\right)$
  - Communication requires $\sqrt{p}$ shifts $O\left(\tau\sqrt{p} + \mu\frac{n^2}{p}\sqrt{p}\right)$
- Total Time: $O\left(\frac{n^3}{p} + \tau\sqrt{p} + \mu\frac{n^2}{\sqrt{p}}\right)$
- **More messages, but Cannon's Algorithm is mei optimal.**

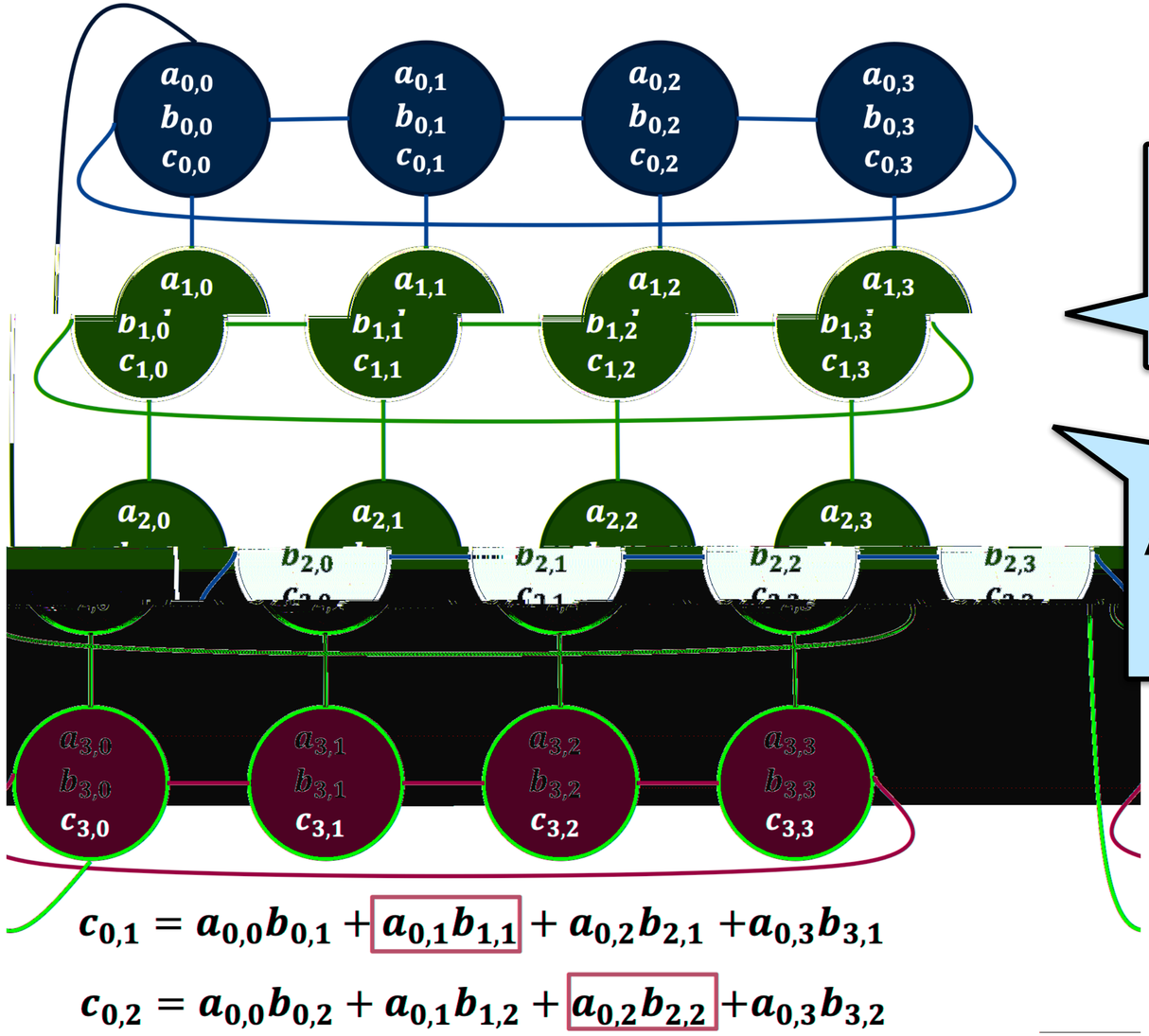
```
1  MatrixMatrixMultiply(int n, double *a, double *b, double *c,
2                       MPI_Comm comm)
3  {
4    int i;
5    int nlocal;
6    int npes, dims[2], periods[2];
7    int myrank, my2drank, mycoords[2];
8    int uprank, downrank, leftrank, rightrank, coords[2];
9    int shiftsource, shiftdest;
10   MPI_Status status;
11   MPI_Comm comm_2d;
12
13   /* Get the communicator related information */
14   MPI_Comm_size(comm, &npes);
15   MPI_Comm_rank(comm, &myrank);
16
17   /* Set up the Cartesian topology */
18   dims[0] = dims[1] = sqrt(npes);
19
20   /* Set the periods for wraparound connections */
```





```
41
42    MPI_Cart_shift(comm_2d, 1, -mycoords[1], &shiftsource, &shiftdest);
43    MPI_Sendrecv_replace(b, nlocal*nlocal, MPI_DOUBLE,
44          shiftdest, 1, shiftsource, 1, comm_2d, &status);
45
46    /* Get into the main computation loop */
47    for (i=0; i<dims[0]; i++) {
48       MatrixMultiply(nlocal, a, b, c); /* c=c+a*b */
49
50       /* Shift matrix a left by one */
51       MPI_Sendrecv_replace(a, nlocal*nlocal, MPI_DOUBLE,
52             leftrank, 1, rightrank, 1, comm_2d, &status);
53
54       /* Shift matrix b up by one */
55       MPI_Sendrecv_replace(b, nlocal*nlocal, MPI_
56             uprank, 1, downrank, 1, comm_2d, &stat
57    }
58
59    /* Restore the original distribution of a and b */
60    MPI_Cart_shift(comm_2d, 0, -mycoords[0],
61    MPI_Sendrecv_replace(a, nlocal*nlocal, M
62          shiftdest, 1, shiftsource, 1, comm_2
63
64    MPI_Cart_shift(comm_2d, 1, -mycoords[1],
65    MPI_Sendrecv_replace(b, nlocal*nlocal,
66          shiftdest, 1, shiftsource, 1, comm_
67
```

Limitations of Cannon's algorithm



Requires a square 2D grid of processors

Also, A and B need to be square

$$c_{0,1} = a_{0,0}b_{0,1} + \boxed{a_{0,1}b_{1,1}} + a_{0,2}b_{2,1} + a_{0,3}b_{3,1}$$

$$c_{0,2} = a_{0,0}b_{0,2} + a_{0,1}b_{1,2} + \boxed{a_{0,2}b_{2,2}} + a_{0,3}b_{3,2}$$

Outer Product Matrix Multiplication

Inner-Product Matrix Multiplication

```
for(int i = 0; i < n; ++i) {
  for(int j = 0; j < n; ++j) {
    for(int k = 0; k < n; ++k) {
      C[i][j] += A[i][k] * B[k][j];
    }
  }
}
```

So far, we have been doing inner product matrix multiply, which calculates n^2 dot products (inner products).

$$C[i, j] = A[i, :] \times B[: , j]$$

Outer product definition

Multiplying two vectors u (n x 1) and v (1 x n) gives us a matrix of size n x n where each entry is the product of one element from u and v.

$$\begin{vmatrix} a \\ b \\ c \end{vmatrix} * \begin{bmatrix} e & f & g \end{bmatrix} = \begin{vmatrix} a*e & a*f & a*g \\ b*e & b*f & b*g \\ c*e & c*f & c*g \end{vmatrix}$$

Outer-Product Matrix Multiplication

```
for(int k = 0; k < n; ++k) {
  for(int i = 0; i < n; ++i) {
    for(int j = 0; j < n; ++j) {
      C[i][j] += A[i][k] * B[k][j];
    }
  }
}
```

The product of a column vector by a row vector is an n x n matrix, or the "multiplication table" of the two elements.

Running over the outer k loop in matrix multiply, we can express C as the **sum of k of these "multiplication table" matrices**.

$$C = \sum_k A[:, k] \times B[k, :]$$

From John Gilbert

SUMMA algorithm

Scalable Universal Matrix Multiplication Algorithm (SUMMA) [van de Geijn and Watts, 97]

- Used in ScaLAPACK, PLAPACK, and PBLAS libraries
- Based on outer product

Avoids need for square matrices and processor grid

```
for(int k = 0; k < n; ++k) {
  C[:,:] += A[:, k] + B[k, :]
}
```

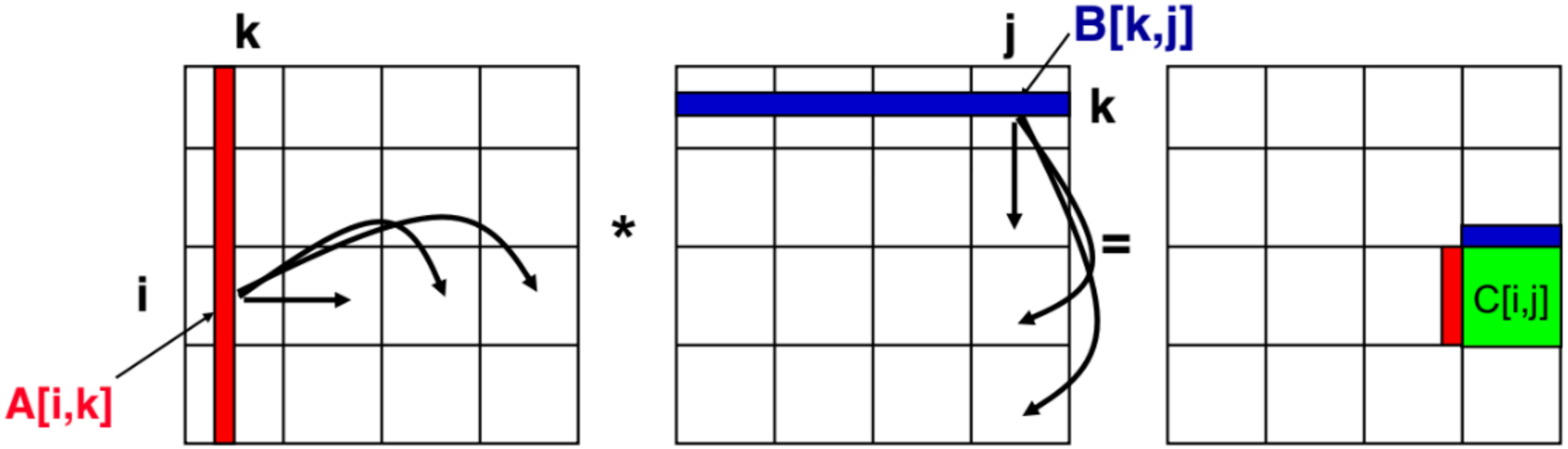
Parallelize all (i, j)

SUMMA algorithm

```
On each process P(i, j):

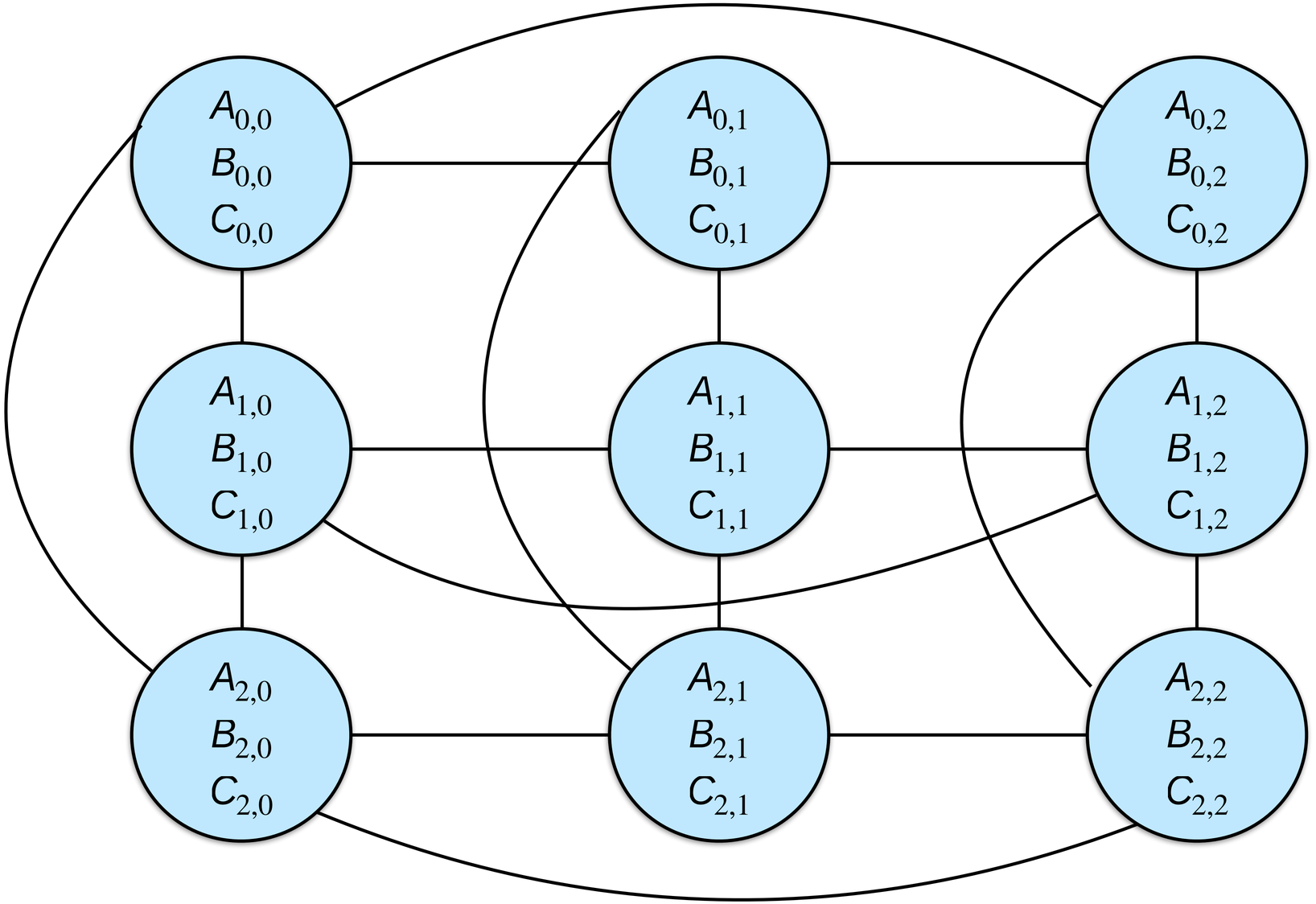
for(int k = 0; k < n; ++k) {
  Bcast column k of A (A[i, k]) within proc row i
  Bcast row k of B (B[k, j]) within proc column j
  Do C += A[i, k] * B[k, j]
}
```

Compute the sum of all outer products



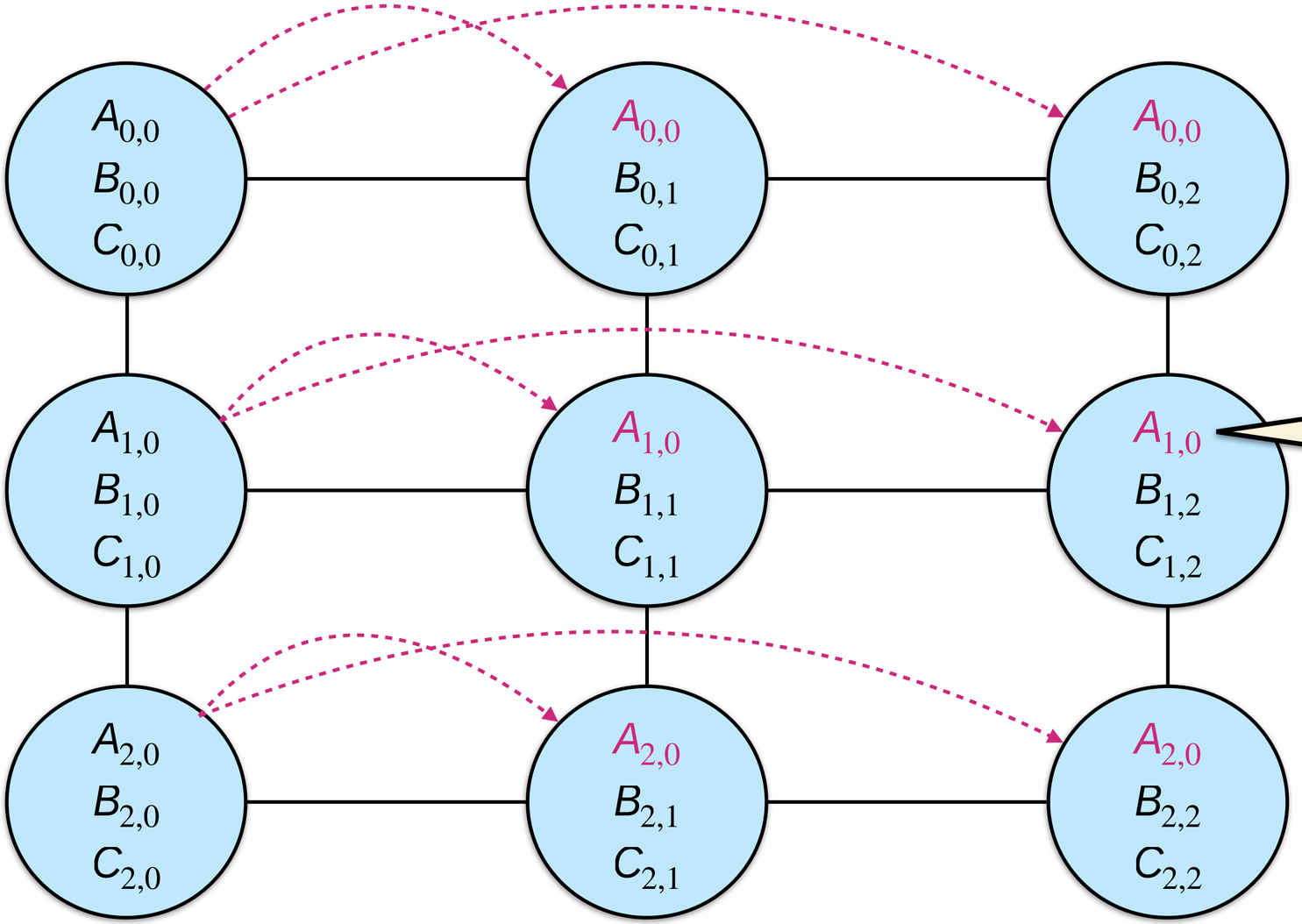
https://cseweb.ucsd.edu/classes/sp11/cse262-a/Lectures/262-pres1-hal.pdf

SUMMA Example: Initial Configuration



SUMMA Example: k = 0

Broadcast $A_{i,0}$ within processor row

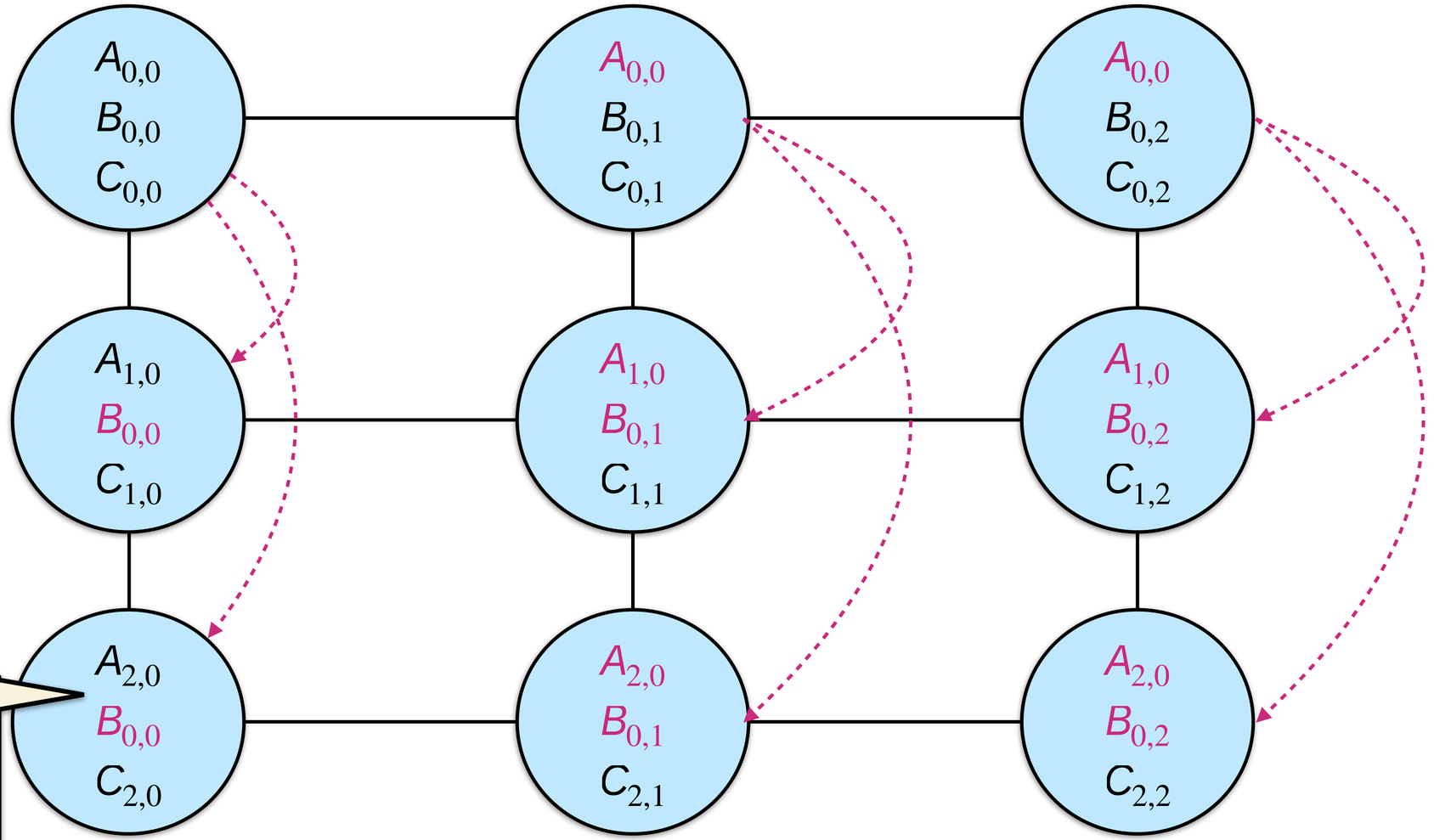


Inner dimension needs to match

keep the original data as well, just not pictured

SUMMA Example: k = 0

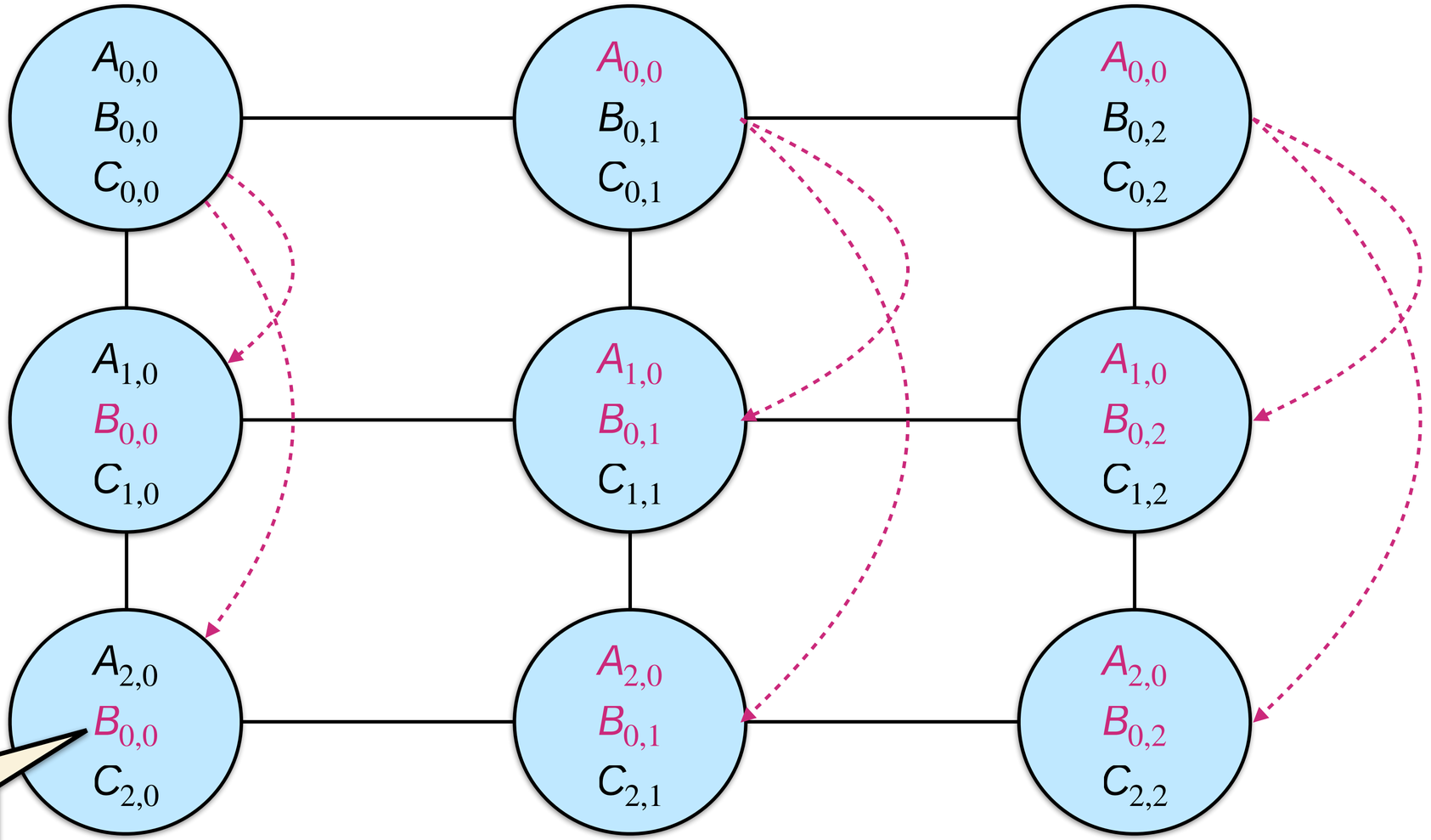
Broadcast $B_{0,j}$ within processor column



Inner dimension matches now: multiply and add to C

SUMMA Example: k = 0

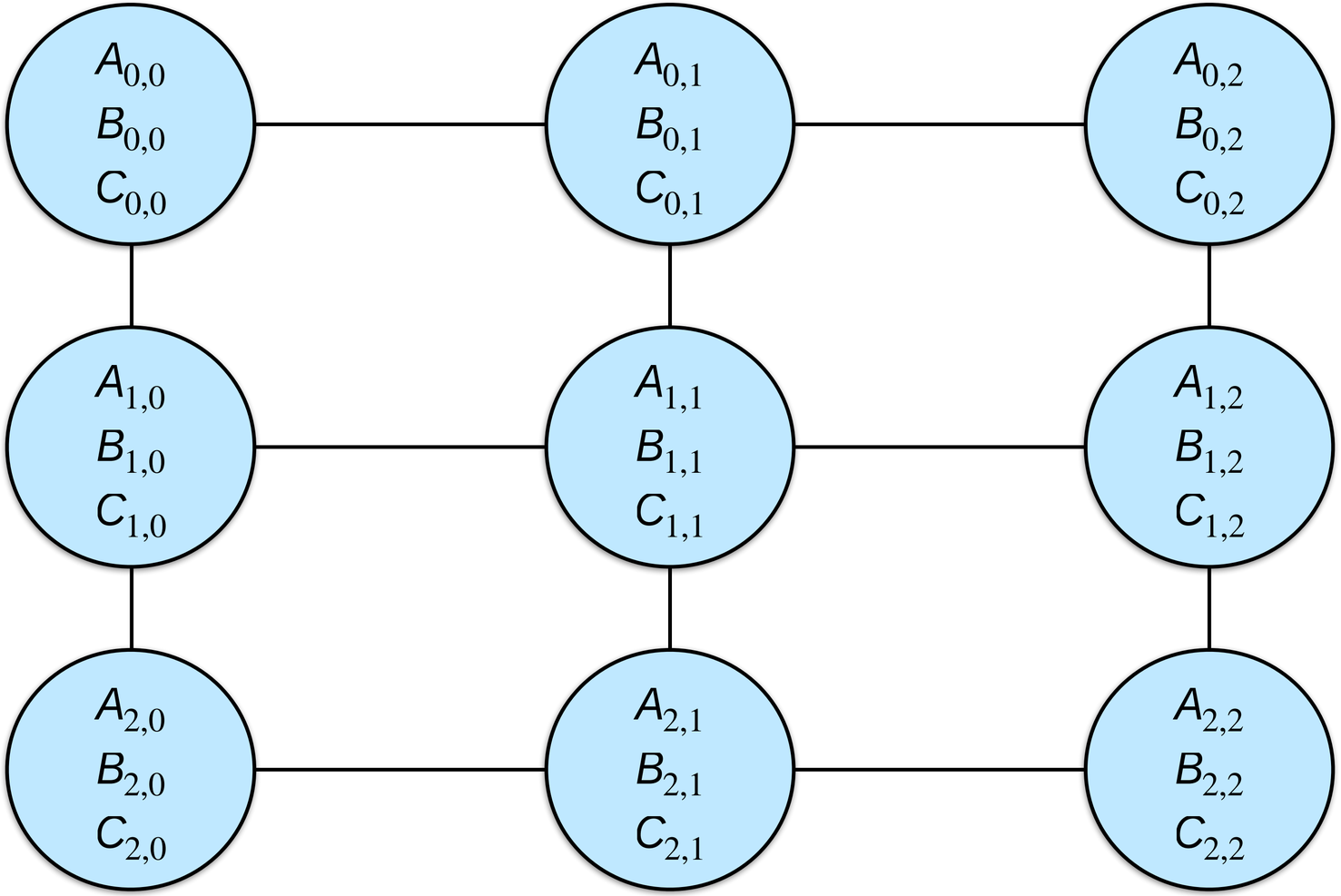
Broadcast $B_{0,j}$ within processor column



Inner dimension matches now

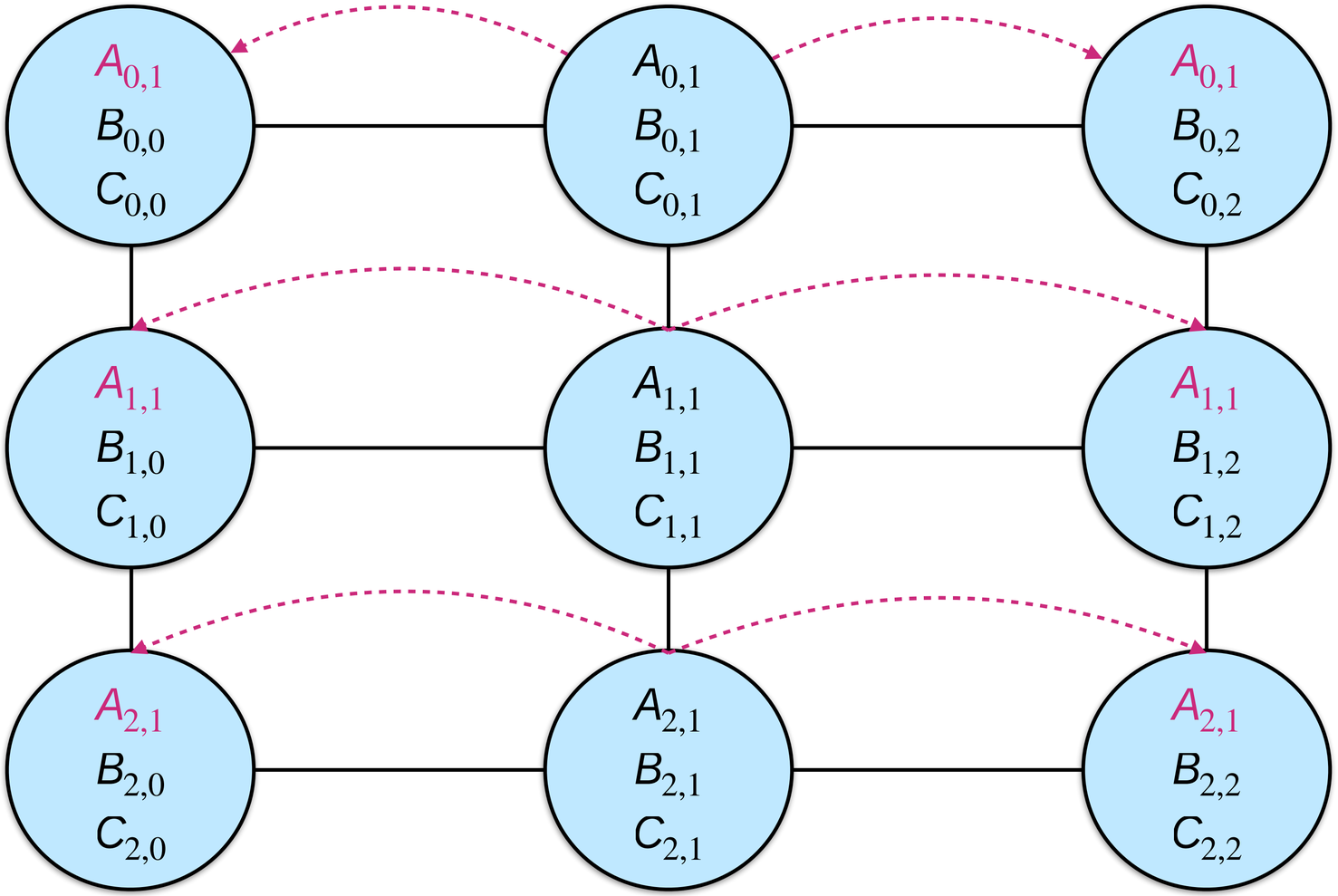
keep the original data as well, just not pictured

Next round: starting with initial configuration again



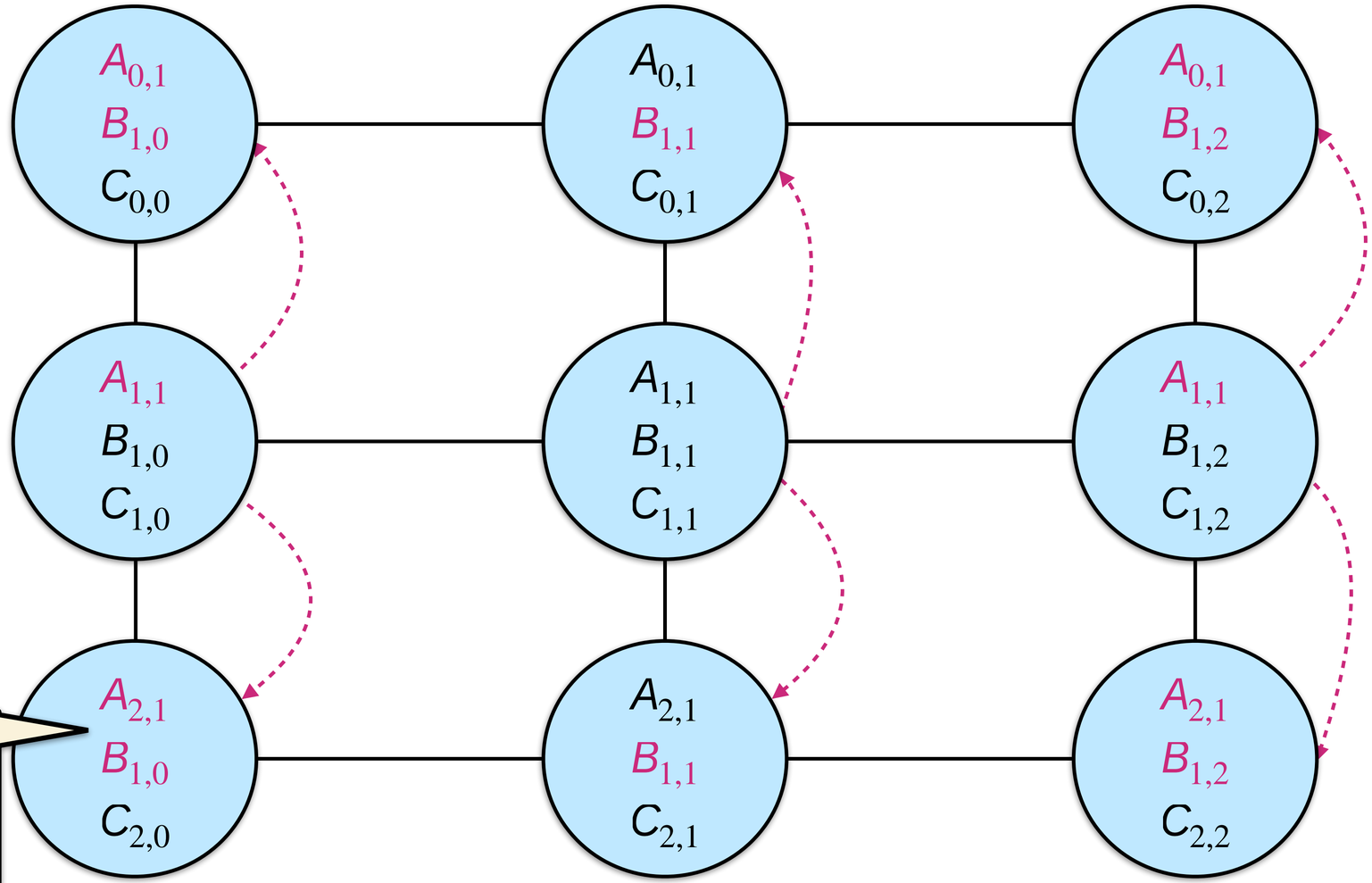
SUMMA Example: k = 1

Broadcast $A_{i,1}$ within processor row



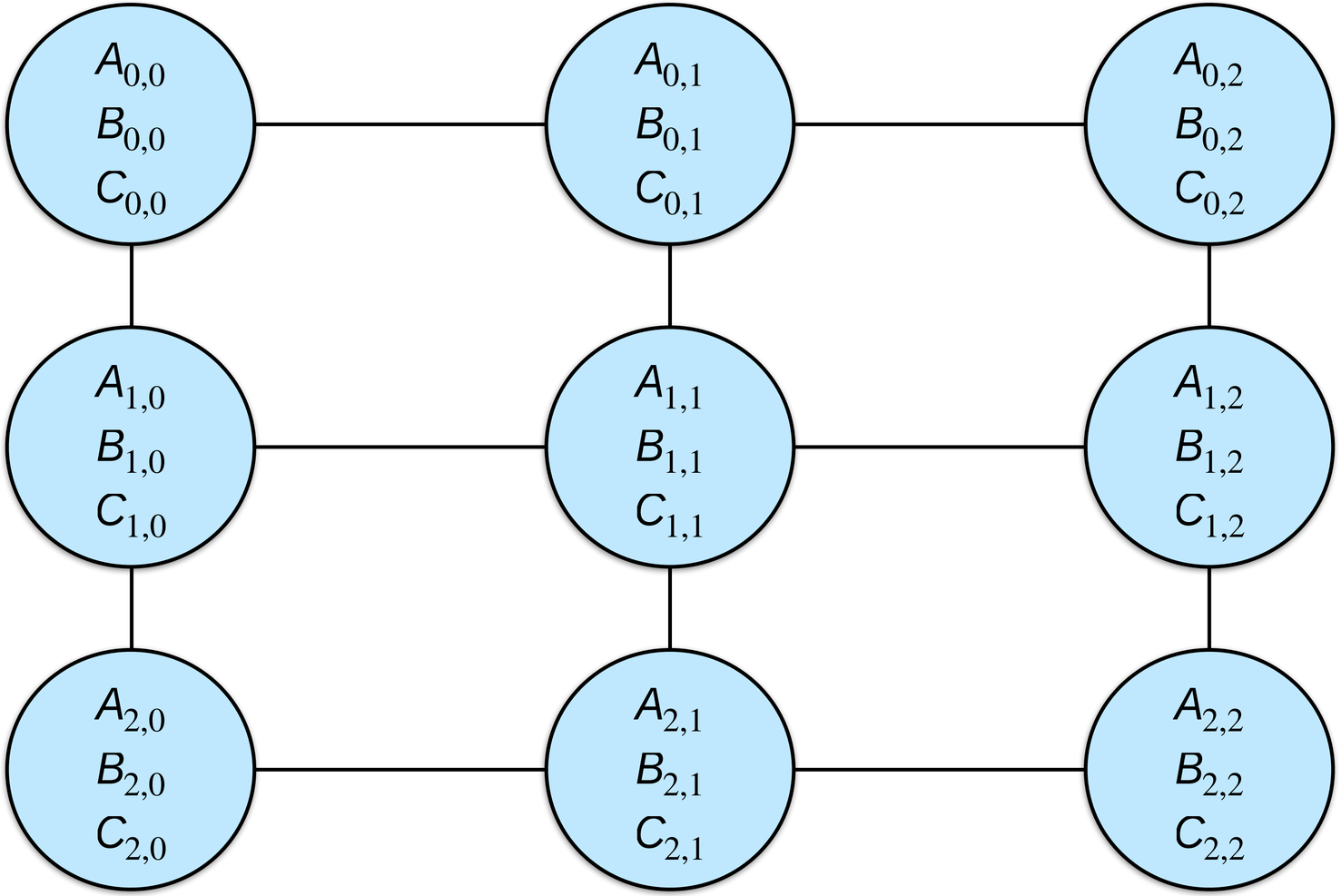
SUMMA Example: k = 1

Broadcast $B_{1,j}$ within processor column

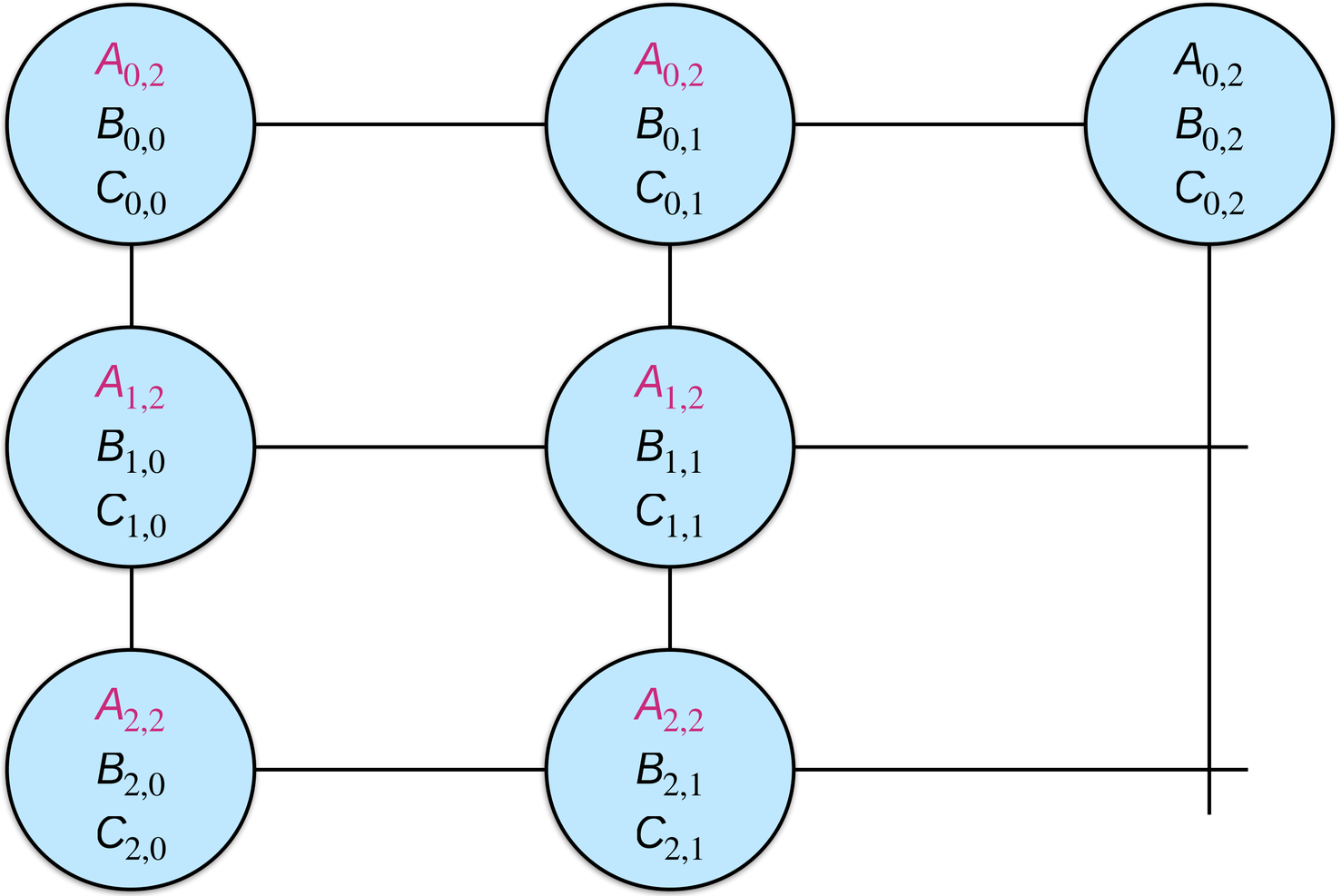


Inner dimension matches now: multiply and add to C

Next round: starting with initial configuration again

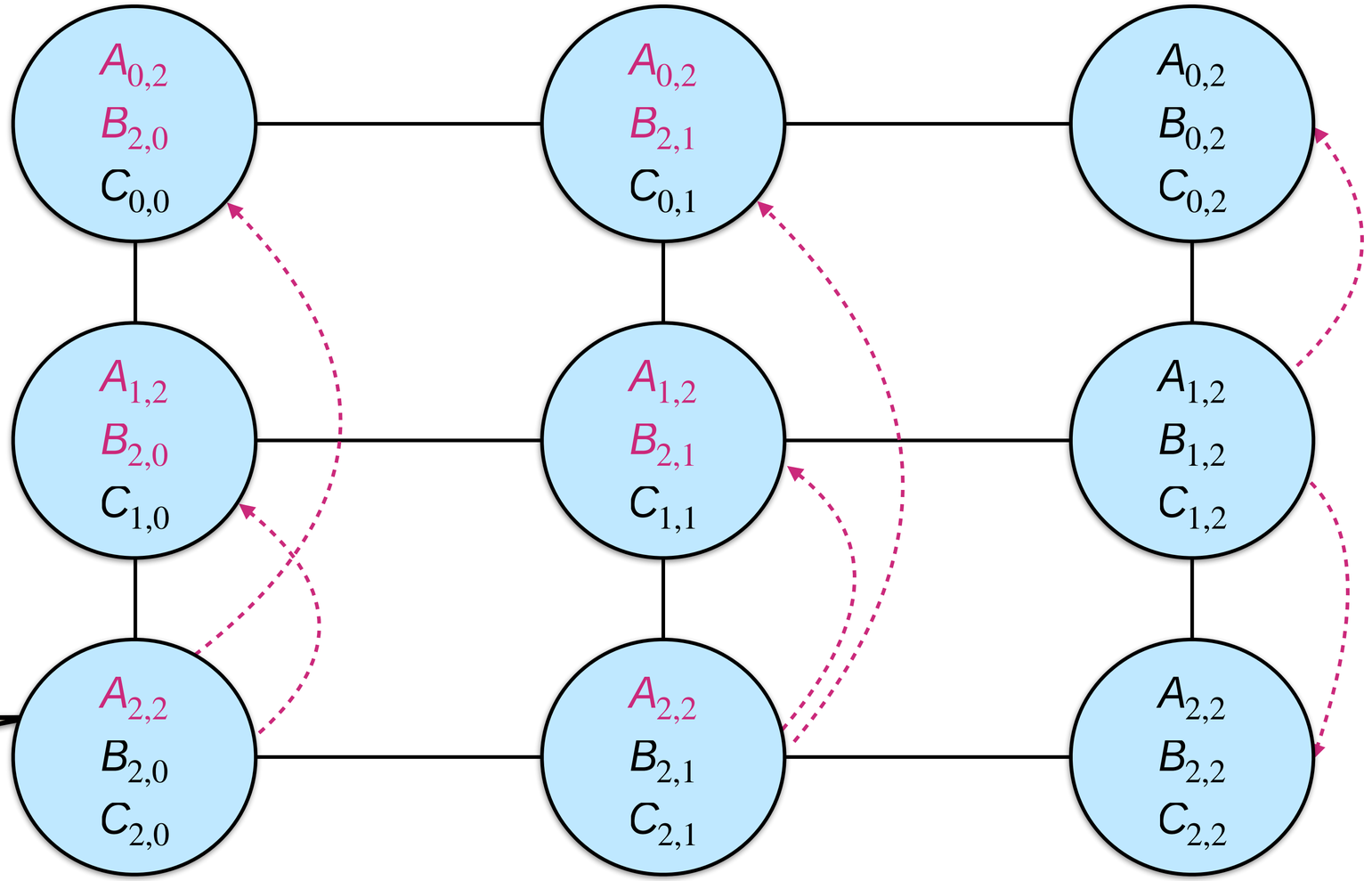


SUMMA Example: k = 2



SUMMA Example: k = 2

Broadcast $B_{2,j}$ within processor column



Inner dimension matches now: multiply and add to C

Blocking matrices in SUMMA

Suppose we have n x n square matrices A, B, C (for simplicity).

Rather than doing n iterations of the outer loop, pick some **block size b** (b columns of A and b rows of B) so that we can do n/b iterations of the outer loop.

```
for(int k = 0; k < n; k+=b) {
  C[:,] += A[:, k:k+b-1] * B[k:k+b-1, :]
}
```

Multiply n x b piece of A with b x n piece of B

Communication cost

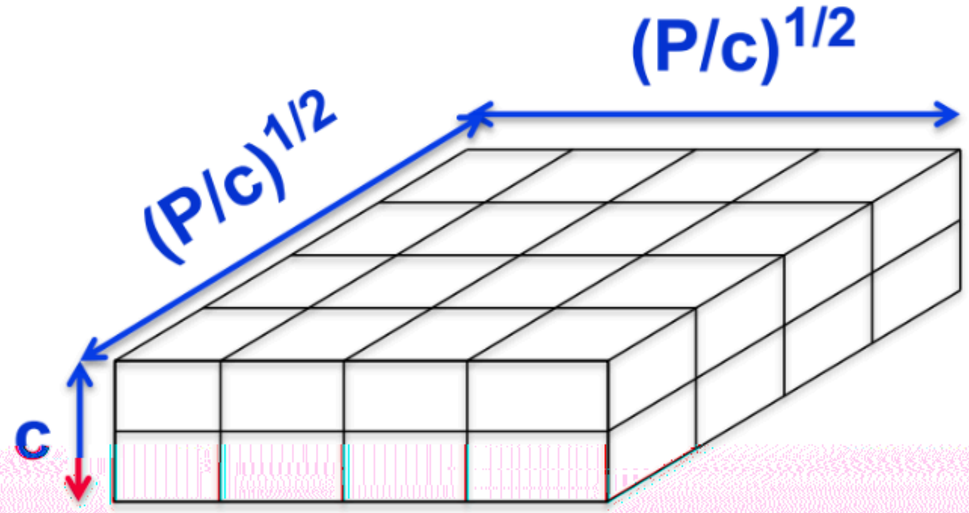
- The simplest choice of block row/col size is $b = O(n/\sqrt{p})$ (assuming p processors in a square grid for simplicity).
- So each submatrix is size $O(n^2/p)$.
- Each broadcast costs $O((\tau + \mu m)\lg(\sqrt{p})) = O((\tau + \mu n^2/p)(\lg p))$
- There are $n/b = O(\sqrt{p})$ rounds
- Total cost is (num rounds) x (cost of broadcast per round)

Substitute $m = n^2/p$

2.5D Matrix Multiplication

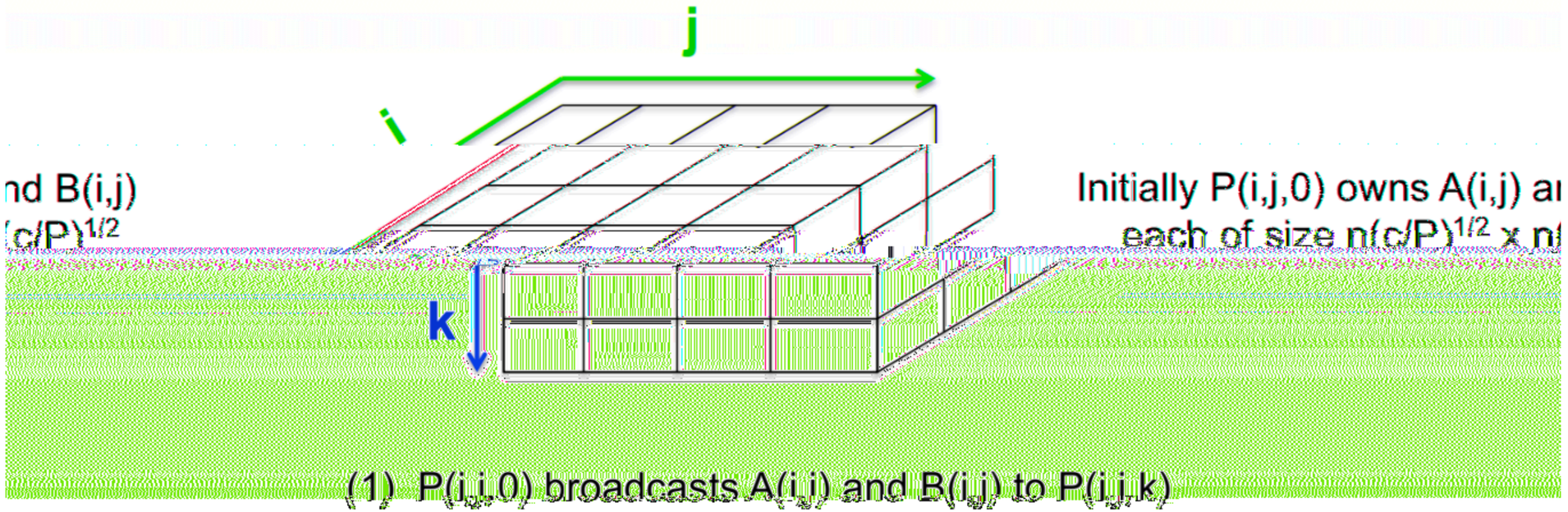
- Assume can fit cn^2/P data per processor, $c > 1$
- Processors form $(P/c)^{1/2}$ x $(P/c)^{1/2}$ x c grid

Example: P = 32, c = 2



2.5D Matrix Multiplication

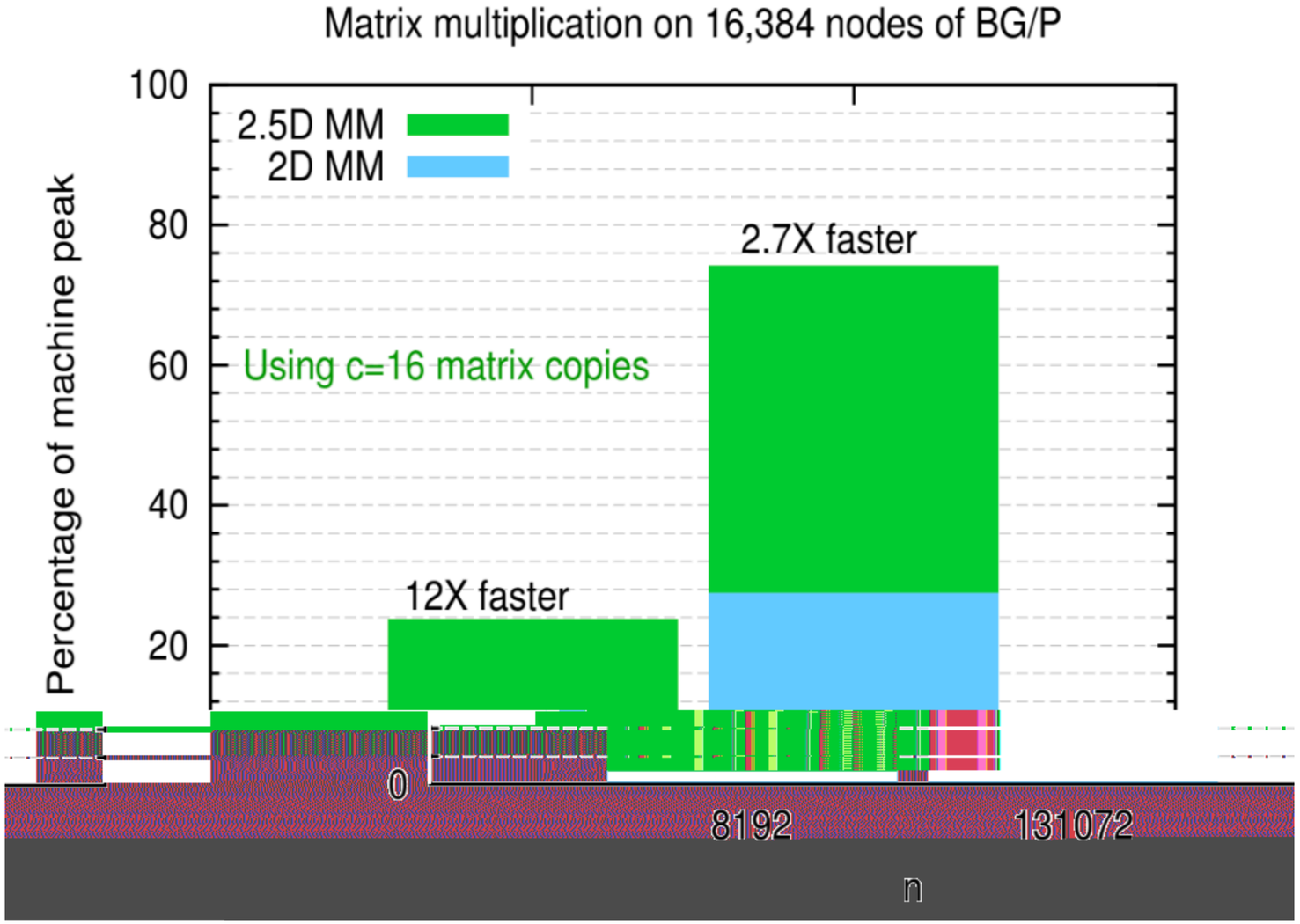
- Assume can fit cn^2/P data per processor, $c \geq 1$
  - Processors form $(P/c)^{1/2}$ x $(P/c)^{1/2}$ x c grid



"Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms" - Solomonik and Demmel, '11

From UC Berkeley CS267

2.5D Matrix Multiplication on IBM BG/P, 16k nodes / 64k cores



"Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms" - Solomonik and Demmel, '11